INDIRA GANDHI NATIONAL OPEN UNIVERSITY

MCSP - 232

LAPTOP PRICE PREDICTION PROJECT

by

Sagar Gupta

Enrolment No: 2451581394

Under Guidance of

Mr. Chandan Kumar Srivastava

Submitted to the School of Computer and Information Sciences, IGNOU in partial fulfilment of the requirements for the award of the degree

Master of Computer Applications (MCA_NEW)

Year Of Submission: 2026

Indira Gandhi National Open University
Maidan Garhi
New Delhi – 110068

ignou
THE PEOPLE'S
UNIVERSITY

इंदिरा गांधी राष्ट्रीय मुक्त विश्वविद्यालय
मैदान गढ़ी, नई दिल्ली - 110068
Indira Gandhi National Open University
Maidan Garhi,New Delhi - 110068

IGNOU - Student Identity Card

2451581394

Enrolment Number :	2451581394
RC Code :	38: DELHI 3 Naraina
Name of the Programme :	MCA_NEW : Master of Computer Applications
Name :	SAGAR GUPTA
Father's Name :	RAKESH KUMAR GUPTA
Address :	C-4 /94A, First floor Keshav PuramNORTH WEST
Pin Code :	110035

Instructions :
1. This card should be produced on demand at the Study Center, Examination Center or any other Establishment of IGNOU to use its facilities.
2. The facilities would be available only relating to the Programme/course for which the student is registered.
3. This ID Card is generated online. Students are advised to take a color print of this ID Card and get it laminated.
4. The student details can be cross checked with the QR Code at www.ignou.ac.in

Registrar
Student Registration Division

SCHOOL OF COMPUTER AND INFORMATION SCIENCES
IGNOU, MAIDAN GARHI, NEW DELHI – 110 068

II. PROFORMA FOR THE APPROVAL OF MCA PROJECT PROPOSAL (MCSP-232)

(Note: All entries of the proforma of approval should be filled up with appropriate and complete information. Incomplete proforma of approval in any respect will be summarily rejected.)

Project Proposal No : M38122501B
(for office use only)

Enrolment No.: 245158139 4
Study Centre: Rajdhani College-38046
Regional Centre: Delhi-3, Narina RC
Code: 38
E-mail: krishanasagar406@gmail.com

Mobile No : 9711651683

1. Name and Address of the Student: Sagar Gupta, C-4/94A, Keshav Puram, New Delhi-110035
2. Title of the Project***: Laptop Price Prediction System
3. Name and Address of the Guide: Mr. Chandan Kumar Srivastava, C-4/99, Keshav Puram, New Delhi-110035

4. Educational Qualification of the Guide: (Attach bio-data also) (*in Computer Science / IT only)	Ph.D*	M.Tech.*	B.E*/B.Tech.*	MCA	M.Sc.*
	☑	☐	☐	☑	☐

5. Working / Teaching experience of the Guide**: 13 Years of experience as a Senior Software Engineer
(**Note: At any given point of time, a guide should not provide guidance for more than 5 MCA students of IGNOU)*

6. Software used in the Project*** : Jupyter Notebook, Python, Streamlit
(*** Please refer to section VIII of these guidelines)

7. Is this your first submission? ☑ Yes ☐ No

Sagar
Signature of the Student
Date: 12-10-2025

Signature of the Guide
Date: 12-10-2025

For Office Use Only

☑ Approved ☐ Not Approved

Name: ....
....
Signature, Designation, Stamp of the Project Proposal Evaluator
Date: .....

Suggestions for reformulating the Project:

CERTIFICATE OF ORIGINALITY

This is to certify that the project report titled **LAPTOP PRICE PREDICTION** submitted to **Indira Gandhi National Open University** in partial fulfilment of the requirement for the award of the degree of **MASTER OF COMPUTER APPLICATIONS,** is an authentic and original work carried out by **Mr. SAGAR GUPTA** with Enrolment No. **2451581394** under my guidance.

The matter embodied in this project is genuine work done by the student and has not been submitted whether to this University or to any other University / Institute for the fulfillment of the requirements of any course of study.

(Signature of the Student)

Date: 24/05/2026

Name and Address
Of the student

Sagar Gupta

C-4/94A, Keshav Puram,

New Delhi, 110035

Enrolment No. 2451581394

(Signature of the Guide)

Date: 24/05/2026

Name, Designation
& Address of the Guide

Mr. Chandan Kumar Srivastava

Senior Software Engineer

C-4/99, Keshav Puram,

New Delhi, 110035

INDIRA GANDHI NATIONAL OPEN UNIVERSITY

Maidan Garhi, New Delhi – 110068
School of Computer and Information Sciences
Phone : 29572902

Project Trainee Letter (MCSP-232)

Date: 24/05/2026

This is to certify that Mr / Ms SAGAR GUPTA with Enrolment No. 245158394 is a final year student of the Master of Computer Applications (Programme Code: MCA_NEW), Indira Gandhi National Open University (IGNOU), and is required to do a final semester project work in his/her final year starting from January/July session. Her / His project must be undertaken in a software development Organization/ Industry/Research Laboratory under the supervision of a guide, preferably from the same organization with the educational qualifications and experience mentioned in the MCSP-232 project guidelines. During her/his course of study, the student has studied and gained knowledge on various Computer Science courses such as Design and Analysis of Algorithms, Object Oriented Analysis and Design, Discrete Mathematics, Accountancy and Financial Management, Computer Networks, Software Engineering, Data Mining and Data Warehousing, Artificial Intelligence and Machine Learning, Data Science and Big Data, Cloud Computing & IoT, Image processing, Mobile Computing. S/he has hands on experience in C programming, Internet Technologies, JAVA, Python, R programming etc. S/he may please be given a chance to work in your esteemed organization and complete her/his project work. I ensure you a sincere and quality output from him. The experience gained by this project work, not only benefit the student to partially fulfil the requirements of the Master of Computer Applications of IGNOU, but also lay a foundation for her/his future career.

Looking forward to your positive response, support and cooperation.

Signature, Name of the Regional Director/ARD/DD with Date and Stamp

(18)

INDIRA GANDHI NATIONAL OPEN UNIVERSITY

School of Computer and Information Sciences (SOCIS) Maidan Garhi, New Delhi – 110068

PROJECT SYNOPSIS

(For Approval of Project Proposal – MCSP-232)

Title of the Project:

Laptop Price Prediction System

Submitted by:

Name of the Student: Sagar Gupta

Enrollment No.: 2451581394

Programme: Master of Computer Applications (MCA_NEW)

Study Centre: Rajdhani College (38046)

Regional Centre: RC DELHI 3, Naraina

Under the Guidance of:

Name of the Guide: Mr. Chandan Kumar Srivastava

Designation: Senior Software Engineer

Organization: CBytes Tech Solutions Pvt. Ltd.

Email: cbytes.info@gmail.com

Mobile: +91 78388 80600

Session: January 2026

Date of Submission: 13/10/2025

Submitted to:

The Regional Director

Indira Gandhi National Open University

Ranganathan Bhawan, C Block, near CGHS Dispensary, Block C, Naraina Vihar, Naraina, New Delhi, Delhi 110028

इंदिरा गांधी राष्ट्रीय मुक्त विश्वविद्यालय
INDIRA GANDHI NATIONAL OPEN UNIVERSITY

Speed Post/ 725
Date:16.12.2025

To,
SAGAR GUPTA
C-4/ 94A,
KESHAV PURAM,
NEW DELHI
110035
MCA-NEW : [MCSP-232]
Enrolment No: [2451581394]

Subject: Return of Evaluated Synopsis- reg.

Dear SAGAR GUPTA,

We hereby inform you that your synopsis has been evaluated by the Evaluator and the same is hereby returned in original for your academic purposes.

If the synopsis is approved, Congratulations. You may now, proceed with the subsequent phases of your research/project as per the guidelines.

If the synopsis not approved, it requires revision as per the Evaluator's comments. Kindly refer to the feedback, and resubmit the revised synopsis within the stipulated timeline.

Best wishes,

Yours Sincerely,

For Project Unit
RC Delhi-3

BIO-DATA OF THE PROJECT GUIDE

Name of the Guide:

Mr. Chandan Kumar Srivastava

Designation:

Senior Software Engineer

Organization Name and Address:

CBytes Tech Solutions Pvt. Ltd.

C-4/99, Block C5, Keshav Puram, Tri Nagar, New Delhi, Delhi, 110035

Mobile No.: +91 78388 80600
Email: Cbytes.info@gmail.com

Educational Qualification:

Degree	Specialization	University	Year of Passing
PHD	Cloud Computing	JNKB, Jabal Pur	Pursuing
MCA	Computer Science	GGSIPU	2013

Professional Experience:

Period	Organization	Designation	Nature of Work
2012-2025	CBytes Tech Solutions Pvt. Ltd.	Senior Software Engineer	Software Development, Teaching, Project Guidance

Total Experience: 13 Years

Areas of Expertise:

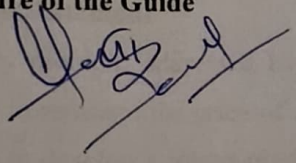
- Software Development
- Database Design
- Web Technologies
- SQL
- Python
- Java
- Power BI
- Machine Learning

Declaration:

I hereby declare that the above information is true to the best of my knowledge and belief. I am willing to guide the MCA student for the project work titled "**Laptop Price Prediction System**" under the IGNOU MCA (MCSP-232) programme.

Date: [12/10/2025]
Place: Keshav Puram, New Delhi

Signature of the Guide



Synopsis

1. Introduction and Problem Statement

1.1. Project Background

The rapid advancement in technology means new laptop models are constantly flooding the market, leading to significant volatility and complexity in determining their true market value. Laptops are complex products, and their price is influenced by a multitude of factors, including the brand, CPU specifications, GPU performance, RAM, storage type (SSD/HDD), screen size, resolution, and operating system. For buyers, the challenge lies in assessing whether a particular laptop's price is justified by its specifications. For sellers, accurately pricing their inventory is critical for competitive advantage. The traditional methods of pricing, often based on rough estimates, outdated price lists, or manual comparison, are inefficient, subjective, and prone to error, leading to dissatisfaction on both sides of a transaction.

1.2 Problem Statement

The primary problem addressed by this project is the lack of a reliable, fast, and data-driven mechanism to estimate the price of a laptop based on its comprehensive technical specifications. The goal is to develop a robust predictive model that can ingest various features of a laptop and output a fair market price estimate, mitigating the ambiguity and uncertainty in the buying and selling process. Furthermore, the solution must be easily accessible, leading to the secondary problem of deploying the machine learning model as a user-friendly, real-time web application.

1.3 Project Objectives

The project aims to achieve the following specific objectives:

1. **Data Acquisition and Pre-processing:** To collect a comprehensive dataset of laptop specifications and their corresponding market prices from multiple credible online sources. This involves extensive cleaning, handling missing values, standardizing categorical features (e.g., brand names, CPU types), and performing necessary data transformations.
2. **Exploratory Data Analysis (EDA):** To conduct in-depth analysis to understand the correlation between various features (such as CPU, GPU, RAM, etc.) and the final price. This includes generating visualisations (histograms, box plots, correlation matrices) to derive meaningful insights and guide the feature engineering process.
3. **Model Development:** To select, implement, and train various regression-based Machine Learning models, with a primary focus on the **Random Forest Regressor**, to predict the continuous price variable. The justification for the Random Forest model is its ability to handle non-linear relationships and its robustness against overfitting.
4. **Model Evaluation and Optimization:** To rigorously evaluate the performance of the trained model using industry-standard metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and R-squared. Hyperparameter tuning will be performed to achieve the highest possible prediction accuracy.
5. **Web Application Development:** To create a dynamic and intuitive user interface using the **Streamlit** framework in Python, allowing any user to input the desired laptop specifications and receive an instant price prediction from the trained model.
6. **Cloud Deployment (New Scope):** To deploy the complete end-to-end Machine Learning web application on a reliable cloud platform, specifically **Heroku**, ensuring 24/7 accessibility, high performance, and scalability for real-world usage. This makes the tool accessible to the public, including buyers, sellers, and market analysts.

2. Present State of Art and Need for the Project

2.1 Present State of Art

Price prediction is a well-established field within machine learning, with applications ranging from stock market forecasting to real estate valuation. Existing approaches for product pricing generally fall into three categories:

1. **Heuristic Methods (Manual/Experience-based):** This relies on expert knowledge and general market trends. While quick, it is highly subjective and lacks accuracy, especially in fast-changing markets.
2. **Simple Statistical Models (Linear Regression):** These models are easy to implement but fail to capture the complex, non-linear relationships between multiple hardware components and the final price. The impact of a premium GPU combined with a low-end CPU, for instance, cannot be effectively modeled linearly.
3. **Advanced Machine Learning Models:** Modern solutions employ techniques like Gradient Boosting Machines (XGBoost, LightGBM), Support Vector Regressors (SVR), and Ensemble methods like Random Forest. These methods, particularly Random Forest, excel at handling high-dimensional data, feature importance ranking, and non-linear interactions, leading to superior prediction accuracy.

2.2 Need for the Project

While numerous studies exist on price prediction (e.g., for cars or houses), a publicly accessible, accurate, and dedicated end-to-end deployed application for laptop price prediction that is both robust in its ML backbone (Random Forest) and user-friendly in its interface (Streamlit) is a vital necessity.

- **For Buyers:** It provides confidence that they are paying a fair price, preventing them from overspending.
- **For Sellers/Dealers:** It enables efficient inventory pricing, speeding up sales cycles and maximizing profit margins based on real-time data.

- **For Market Analysts:** The model's feature importance analysis can offer valuable insights into which specifications are driving the current market price trends.
- **For Academic Learning:** The project provides a comprehensive, practical experience in the entire Data Science lifecycle, from data collection and model building to web deployment, fulfilling a core requirement of the MCA program.

3. Project Methodology and Planning

The project will follow the standard CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology, adapted for an end-to-end Machine Learning project.

3.1 Phase 1: Business Understanding and Data Collection (4 Weeks)

- **Objective:** Define key features that influence laptop prices (Brand, CPU, GPU, RAM, Storage, Screen Size, OS).
- **Action:** Scrape/Collect a large, diverse dataset from reputable e-commerce platforms or public repositories.
- **Deliverable:** A raw, multi-featured dataset (CSV format).

3.2 Phase 2: Data Preprocessing and EDA (6 Weeks)

- **Objective:** Clean the data and gain insights.
- **Action:**
    - Handle missing values (imputation or removal).
    - Perform Data Cleaning: Standardise units (e.g., convert all RAM/Storage to GB).
    - **Feature Engineering:** Extract numerical values from complex string features (e.g., separating CPU speed, core count, or GPU VRAM).
    - **EDA:** Use visualisations (Matplotlib, Seaborn) to analyze correlations.
- **Deliverable:** Cleaned, processed, and engineered dataset ready for modeling.

3.3 Phase 3: Model Development and Evaluation (8 Weeks)

- **Objective:** Train, tune, and select the best model.
- **Action:**
    - Split data into training and testing sets.
    - Apply encoding (One-Hot Encoding) to categorical features.
    - Train **Random Forest Regressor** (primary model) and other contenders (Linear Regression, SVR) for comparison.
    - **Hyperparameter Tuning:** Use Grid Search or Random Search to find optimal parameters for the Random Forest model.
    - **Evaluation:** Calculate R-squared, MAE, and MSE on the test set.
    - **Persistence:** Save the final, trained model and the required transformation objects (e.g., One-Hot Encoder, Scaler) using Python's `pickle` library.
- **Deliverable:** Final serialized model file (`.pkl`) and a comprehensive evaluation report.

3.4 Phase 4: Web Application Development and Deployment (8 Weeks)

- **Objective:** Create a functional web application and deploy it to the cloud.
- **Action:**
    - **Frontend Development:** Build the input form and prediction display using **Streamlit**.
    - **Backend Integration:** Create a Python script that loads the pickled model and runs the prediction function upon user input.
    - **Deployment Setup:** Create necessary deployment files (e.g., `requirements.txt`, `Procfile`).
    - **Cloud Deployment:** Deploy the entire application repository to **Heroku**.
- **Deliverable:** Fully functional, publicly accessible web application.

4. System Requirements Analysis

A detailed system requirements analysis is critical to ensure the smooth development and operation of the end-to-end application.

4.1 Hardware Requirements

The hardware requirements are divided into two parts: the development system and the deployment environment.

Component	Minimum Specification (Development)	Recommended Specification (Development)
Processor	Dual-Core, 2.0 GHz or higher	Intel Core i5/i7 (8th Gen or higher) or equivalent AMD
RAM	4 GB	8 GB or 16 GB (for faster data processing and model training)
Storage	100 GB Free HDD Space	256 GB SSD (for faster I/O operations)
Internet	Stable Broadband Connection (for data collection and deployment)	High-speed connection
Deployment	N/A (Handled by Heroku's infrastructure)	Heroku Dynos (Standard 1x/2x for production load)

The model training and data analysis are CPU and RAM-intensive. While training on a local machine is feasible, the final deployment on **Heroku** will leverage its scalable infrastructure, ensuring the web application remains responsive.

4.2 Software Requirements

The project relies exclusively on open-source software and Python libraries, ensuring low cost and high flexibility.

Category	Component	Justification and Role in Project
Operating System	**Windows, Linux, or macOS**	The entire codebase is cross-platform. Linux (Ubuntu) is preferred for deployment to Heroku.
Programming Language	**Python (version 3.8+)**	The primary language for ML, data analysis, and web development (via Streamlit).
ML Frameworks	**scikit-learn**	Provides robust, efficient implementation of the Random Forest Regressor and all necessary preprocessing tools.
Data Handling	**Pandas, NumPy**	Essential for data manipulation, cleaning, and numerical operations on the dataset.
Visualization	**Matplotlib, Seaborn**	Used during the EDA phase to generate informative charts and graphs.

Web Framework	**Streamlit**	Chosen specifically for its rapid development capabilities, allowing creation of interactive data apps with pure Python.
Version Control	**Git**	For tracking changes, collaboration, and integration with the Heroku deployment pipeline.
Cloud Platform	**Heroku**	The chosen Platform-as-a-Service (PaaS) for deploying the Streamlit application via Git.
Deployment Tools	**Gunicorn/Waitress (for Heroku)**	Necessary WSGI servers to run the Streamlit application in a production environment on the cloud.
IDE	**VS Code or Jupyter Notebook**	For writing, testing, and debugging the Python code.

5. Detailed System Architecture and Design

The proposed system follows a classic **Three-Tier Architecture** (Data Tier, Logic/Application Tier, Presentation Tier) but is specifically tailored for an MLOps (Machine Learning Operations) flow.

5.1 System Architecture Diagram

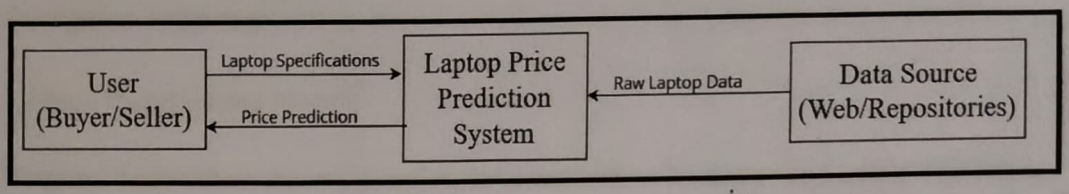
The system architecture is structured to facilitate a clear flow from raw data to a real-time prediction service.

Flow Description:

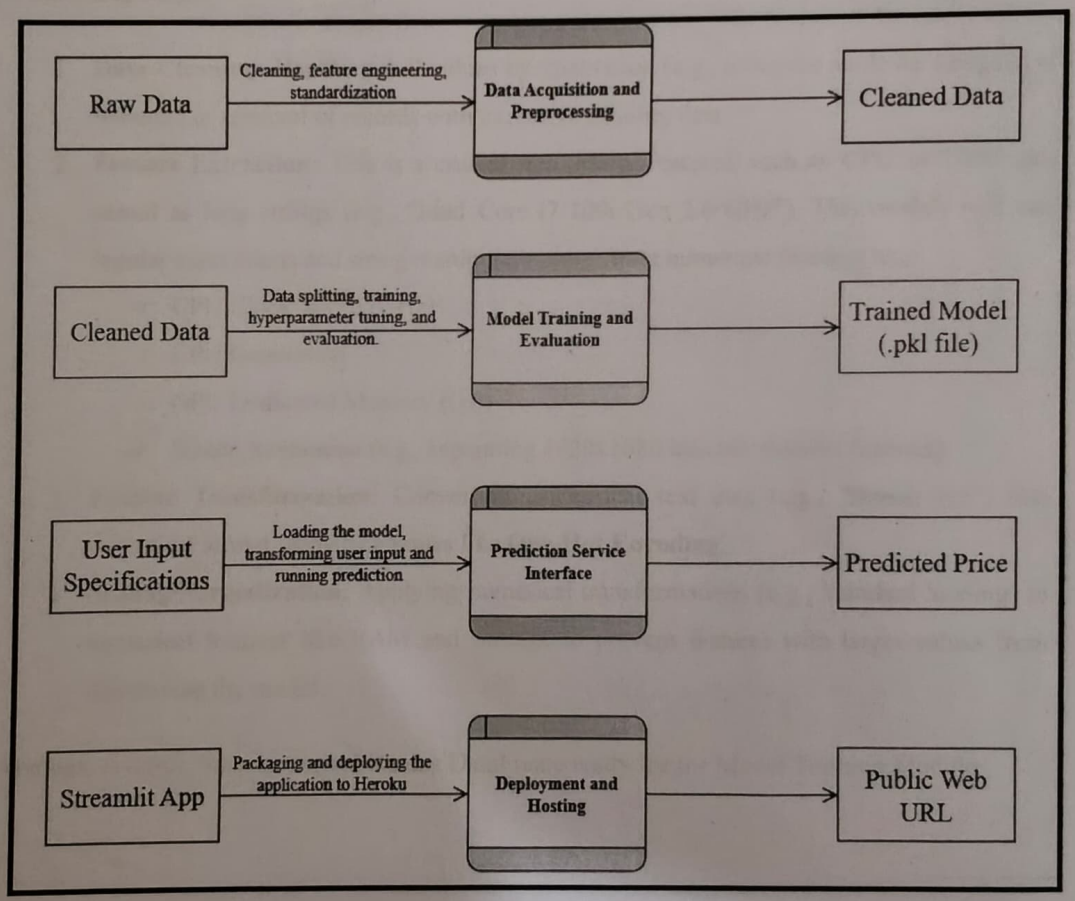
1. **Data Source:** Raw, unstructured laptop data is collected from external repositories.
2. **Data Processing Pipeline (Local/Development):** This involves the heavy lifting of cleaning, feature engineering, and splitting the data.
3. **Model Training:** The processed data is fed into the **Random Forest Regressor** to train the prediction model.
4. **Model Persistence:** The final model and required transformation objects are serialized into a `.pkl` file. This acts as the **Model Repository**.
5. **Web Application (Streamlit):** The Streamlit app loads the persisted `.pkl` model and provides the user interface. It acts as the prediction API server.
6. **Cloud Deployment (Heroku):** The Streamlit application, along with its dependencies and the pickled model, is deployed to Heroku, making it accessible via a public URL.
7. **User Interaction:** A user interacts with the Heroku-deployed web interface, inputs data, and receives a real-time price prediction.

5.2 Data Flow Diagram (DFD)

5.2.1 DFD Level 0 (Context Diagram)



5.2.2 DFD Level 1 (High-Level Processes)



6. Module Description

The entire project is broken down into four distinct, cohesive modules, allowing for independent development and testing.

6.1 Data Preprocessing Module

Purpose: To transform the raw, complex, and potentially noisy dataset into a structured format suitable for machine learning algorithms.

Input: Raw CSV file containing heterogeneous laptop data.

Processing Steps:

1. **Data Cleaning:** Handling null values by imputation (e.g., using the mode for categorical features) or removal of records with excessive missing data.
2. **Feature Extraction:** This is a crucial step. Many features, such as 'CPU' or 'GPU', are stored as long strings (e.g., "Intel Core i7 10th Gen 2.6 GHz"). This module will use regular expressions and string manipulation to extract numerical features like:
    - CPU Clock Speed (GHz)
    - CPU Generation
    - GPU Dedicated Memory (GB)
    - Screen Resolution (e.g., separating 1920x1080 into two distinct features)
3. **Feature Transformation:** Converting categorical text data (e.g., "Brand: HP") into numerical format using techniques like **One-Hot Encoding**.
4. **Scaling/Normalization:** Applying numerical transformations (e.g., Standard Scaling) to numerical features like RAM and Storage to prevent features with larger values from dominating the model.

Output: A clean, fully numerical Pandas DataFrame ready for the Model Training Module.

6.2 Model Training and Persistence Module

Purpose: To develop and finalize the predictive model and save it for future use by the web application.

Input: Cleaned and processed dataset from the Data Preprocessing Module.

Processing Steps:

1. **Data Split:** Dividing the dataset into training (e.g., 80%) and testing (e.g., 20%) sets.
2. **Model Selection:** Implementing the **Random Forest Regressor**. Random Forest is chosen because it is an ensemble method that aggregates the results of multiple decision trees, which reduces variance and generally leads to higher predictive accuracy on complex, real-world data like this.
3. **Training:** Fitting the Random Forest model to the training data.
4. **Hyperparameter Tuning:** Fine-tuning critical hyperparameters (e.g., `n_estimators`, `max_depth`, `min_samples_leaf`) using cross-validation to maximize the R-squared value while minimizing the Mean Absolute Error (MAE).
5. **Persistence:** Serializing the final, optimized model and all necessary preprocessing objects (e.g., the One-Hot Encoder used for categorical data) into separate `.pkl` files.

Output: `model.pkl` (the trained Random Forest model) and `preprocessor.pkl` (the objects needed to transform new user input).

6.3 Prediction API Module

Purpose: To serve the trained model as a functional backend prediction service. In the context of Streamlit, this is realized as a dedicated function within the main application script.

Input: A dictionary or list of laptop features provided by the user via the web interface.

Processing Steps:

1. **Load Assets:** The module first loads the `model.pkl` and `preprocessor.pkl` files into memory.

2. **Input Validation and Transformation:** It ensures the user input matches the expected schema of the training data. It then uses the loaded `preprocessor` to transform the new raw input into the exact numerical format expected by the `model.pkl` (i.e., applying the same feature engineering and one-hot encoding used during training).
3. **Prediction:** The transformed numerical vector is passed to the loaded Random Forest model's `.predict()` method.
4. **Result Formatting:** The predicted price (which may be a floating-point number) is rounded and formatted into a user-readable currency string.

Output: The predicted price of the laptop.

6.4 Web Interface (Streamlit) Module

Purpose: To provide a simple, interactive, and visually appealing front-end application for the end-user.

Input: User selections/inputs for all relevant laptop features (e.g., Brand, RAM size, CPU model, etc.).

Processing Steps:

1. **Library Import and Setup:** Setting up the Streamlit app layout.
2. **User Input Widgets:** Creating interactive widgets for every feature in the model:
    - Dropdown menus (`st.selectbox`) for categorical features (Brand, OS, Type).
    - Number inputs (`st.number_input`) for numerical features (RAM, Storage size, Screen size).
    - A primary **Predict** button (`st.button`) to trigger the prediction logic.
3. **Interaction Logic:** When the user clicks the **Predict** button, the inputs are collected and passed to the Prediction API Module (Section 6.3).
4. **Output Display:** The returned predicted price is displayed prominently to the user with appropriate formatting and context.

5. **Information Display:** The interface will also include sections for displaying general project information, model performance metrics (e.g., the R-squared score), and a description of the project.

Output: A live, interactive web application accessible through the browser.

7. Implementation Details: Deployment Strategy

A successful machine learning project requires not only an accurate model but also a robust, accessible deployment. This project leverages the modern MLOps approach using Streamlit for rapid web development and Heroku for scalable cloud hosting.

7.1 Streamlit for Frontend

Streamlit is chosen as the frontend framework due to its seamless integration with the Python data science ecosystem.

- **Speed of Development:** Streamlit allows the entire web application to be written in pure Python, eliminating the need for complex HTML, CSS, and JavaScript, dramatically reducing development time.
- **Interactivity:** It handles state management and widget interaction natively, ensuring a smooth and responsive user experience.
- **Model Integration:** It easily loads and runs the pickled Python model (`.pkl`), making the integration of the Prediction API Module straightforward.
- **Data Visualization:** Streamlit's built-in charting functionalities will be used to display sample data distributions or model evaluation charts directly within the application's sidebars.

The main application file (`app.py`) will contain all the necessary logic for loading the model, accepting user input, running the prediction, and displaying the results.

7.2 Heroku for Cloud Deployment

Heroku is chosen as the Platform-as-a-Service (PaaS) to deploy the Streamlit application.

- **Ease of Deployment:** Heroku supports Git-based deployment, meaning the application is deployed simply by pushing the code to the Heroku remote repository.
- **Requirements Management:** Heroku automatically detects the Python environment and uses the `requirements.txt` file to install all necessary libraries (Pandas, scikit-learn, Streamlit, etc.).
- **Procfile Configuration:** A simple `Procfile` will be created to instruct Heroku on how to start the Streamlit application using a web server like `gunicorn` or `waitress` to ensure it runs correctly in the cloud environment.
    - **Example `Procfile` content:** `web: sh setup.sh && streamlit run app.py` (where `setup.sh` prepares the environment if needed).
- **Scalability:** Heroku's Dynos (containers) provide a robust, scalable environment, ensuring the application can handle concurrent users and remain responsive, fulfilling the objective of real-world accessibility.

This combination of Streamlit and Heroku provides an efficient, low-overhead path to achieving a fully functional, public-facing machine learning web service.

8. Database/Data Storage Structure

8.1 Data Storage

The project does not require a traditional relational or NoSQL database for real-time transactions. The primary data storage needs are:

1. **Input Dataset:** The raw and cleaned data will be stored as `.csv` files during the development phase.

2. **Trained Model Assets:** The core of the system, the trained Random Forest model and the preprocessing objects (encoders, scalers), will be stored as binary files using Python's `pickle` library. These files will be bundled with the application code and loaded into memory when the Streamlit application starts on Heroku.

8.2 Data Schema (Example Features)

The final dataset, after preprocessing, will consist of numerous numerical features derived from the raw data.

Feature Name	**Data Type**	**Description (Example Value)**	**Encoding Type**
Price (Target)	Float	The sale price of the laptop (e.g., 85000.00)	N/A (Continuous Target)
Brand_HP	Binary (0/1)	One-Hot Encoding for Brand	OHE
Brand_Dell	Binary (0/1)	One-Hot Encoding for Brand	OHE
RAM_GB	Integer	Total RAM size in GB (e.g., 8, 16)	Standard Scaled
Storage_GB	Integer	Total storage size in GB (e.g., 256, 512)	Standard Scaled

Storage_Type_SSD	Binary (0/1)	Type of storage used	OHE
CPU_i7	Binary (0/1)	Indicator for a Core i7 processor	OHE
CPU_Clock_Speed	Float	Numerical clock speed in GHz (e.g., 2.3)	Standard Scaled
Screen_Size_Inches	Float	Diagonal screen size (e.g., 15.6)	Standard Scaled
GPU_VRAM_GB	Integer	Dedicated Graphics VRAM in GB (e.g., 4)	Standard Scaled
OS_Windows	Binary (0/1)	Operating system indicator	OHE

This structured format ensures that the model can interpret all inputs accurately.

9. Limitations of the Project

While the project is designed to be robust, certain limitations must be acknowledged:

1. **Data Currency and Market Volatility:** The accuracy of the model is inherently tied to the dataset's age. Laptop prices change rapidly due to new releases, discounts, and economic factors. The model may suffer from "data drift" over time and would require retraining with a more current dataset to maintain peak performance.
2. **Feature Scope:** The model is limited to the features present in the dataset (e.g., CPU, RAM, GPU, etc.). Intangible factors like aesthetics, specific software bundles, manufacturer warranty status, or consumer reviews (which are hard to quantify) are not considered and can affect real-world pricing.
3. **Geographic Bias:** The collected data will likely be skewed towards a specific market (e.g., Indian or US market). Applying the model to a vastly different geographical area may lead to inaccurate predictions due to varying taxes, import duties, and local pricing strategies.
4. **Random Forest Interpretability:** Although Random Forest provides feature importance, it is generally less interpretable than simpler models like Linear Regression. It can be harder to explain *why* the model made a specific prediction (though its superior accuracy usually justifies this trade-off).
5. **Heroku Free Tier Constraints:** While Heroku deployment is robust, using the free tier for initial deployment may impose limitations on concurrent usage and monthly uptime hours, which could be a factor if the application were to experience high traffic.

10. Future Scope of the Project

The project provides a strong foundation for future development and expansion, including:

1. **Dynamic Pricing Model:** Implement an automated retraining pipeline. This would involve connecting the system to a dynamic data source (e.g., a continuous web scraping agent) and scheduling weekly or monthly retraining of the model on the latest data. This addresses the limitation of data currency.
2. **Advanced Model Exploration:** Experiment with Deep Learning models, such as **Recurrent Neural Networks (RNNs)** or **Transformers**, which are often employed in sequence-to-sequence prediction or complex structured data to potentially achieve even higher accuracy than the Random Forest model.
3. **Sentiment Analysis Integration:** Incorporate an additional module to perform web scraping on product reviews. A Sentiment Analysis model could process this text data and generate a "Sentiment Score" (e.g., -1.0 to 1.0) to be used as an input feature for the prediction model. This would allow the model to capture the value of consumer perception.
4. **Personalized Recommendations:** Develop a user authentication feature (using Firebase or similar services). The system could then track a user's search history and preferences to offer personalized laptop price predictions and recommendations.
5. **Market Analysis Dashboard:** Enhance the Streamlit interface to include advanced data visualization tools that allow the user to analyze market trends, compare predicted vs. actual prices, and view feature importance plots in real-time.
6. **Containerization and Scalability:** Migrate the Heroku deployment to a more robust, scalable solution like **Docker/Kubernetes** on cloud platforms like AWS, GCP, or Azure. This would ensure the application can handle enterprise-level loads.

11. References and Bibliography

This project draws upon established literature in the fields of machine learning, statistical modeling, and web application development.

1. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. (Foundation for the core predictive algorithm).
2. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems* (2nd ed.). O'Reilly Media. (Reference for model implementation and evaluation metrics).
3. McKinney, W. (2017). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython* (2nd ed.). O'Reilly Media. (Used for data manipulation and preprocessing techniques).
4. Grus, J. (2019). *Data Science from Scratch: First Principles with Python* (2nd ed.). O'Reilly Media. (Theoretical background for data science concepts).
5. Streamlit Documentation. (n.d.). Retrieved from https://docs.streamlit.io (Official documentation for the web framework).
6. Heroku Dev Center. (n.d.). Retrieved from https://devcenter.heroku.com (Official documentation for the cloud deployment platform).
7. Public Dataset Source/Repository URL (https://github.com/campusx-official/laptop-price-predictor-regression-project/blob/main/laptop_data.csv).

LAPTOP PRICE PREDICTION

PROJECT REPORT

CHAPTER 1: Objective & Scope of the Project

1.1 Objective of the Project:

The main objective of the "Laptop Price Prediction" project is to employ machine learning techniques to accurately predict the price of laptops based on various features. This project seeks to contribute to the automotive industry by providing a reliable and efficient tool that assists both buyers and sellers in determining fair and competitive prices for used laptops. The primary focus includes data analysis, model development, and the creation of a user-friendly interface for seamless predictions.

In today's rapidly evolving technological landscape, the ability to harness data and use it to derive meaningful insights has become an invaluable skill. Data-driven decision-making is no longer a competitive edge but rather a necessity. This paradigm shift extends to various domains, and one area where data analysis plays a pivotal role is in the automobile industry, particularly in the buying and selling of laptops.

The process of buying or selling a laptop, while seemingly straightforward, is often plagued by a fundamental challenge: pricing. Both buyers and sellers aspire to strike a deal that reflects the true value of the vehicle, creating a win-win scenario. However, determining this value is often complex and riddled with uncertainty. It's a challenge that impacts the automotive market, as well as laptop enthusiasts, dealerships, and ordinary consumers alike.

The issue at the heart of this project is the accurate valuation of automobiles. Buyers want to ensure they are making an informed purchase without overpaying, while sellers aim to set a fair price that attracts buyers without undervaluing their assets.

Our Project addresses this critical issue by creating a solution that leverages machine learning and data analysis techniques. Through the collection and analysis of a diverse datasets comprising laptops from various manufacturers, we have endeavored

to develop a system that accurately predicts the market price of a vehicle based on a set of key attributes. This not only streamlines the process for both buyers and sellers but also provides valuable insights into market trends for analysts and enthusiasts.

Over the course of our project, we have conducted data cleaning and analysis, applied data visualization techniques to extract meaningful patterns, and implemented machine learning models to make precise price predictions.

This project stands at the intersection of data science and the automobile industry, offering a tangible and innovative solution to a longstanding problem. The project unfolds as a journey through data collection, analysis, modeling, and application development. It encapsulates the power of data-driven decision-making and presents an elegant solution to an everyday challenge.

1.2 Scope of the Project:

The scope of the "Laptop Price Prediction" project encompasses various aspects related to the prediction of laptop prices using machine learning techniques. Here is an expanded view of the project scope:

1. Data Collection and Cleaning: Gather a diverse dataset containing information on laptops, including features such as manufacturer, model, production year, category, display type, etc. Perform data cleaning to handle missing values and outliers and ensure data quality.
2. Data Analysis and Visualization: Conduct exploratory data analysis to understand the distribution of data and identify patterns. Utilize data visualization techniques such as graphs and charts to present insights derived from the dataset.
3. Machine Learning Model Development: Employ machine learning algorithms, such as Random Forest, Logistic Regression, and Linear Regression, for predicting Laptop prices. Train and fine-tune these models using the dataset to enhance predictive accuracy.
4. Feature Engineering: Explore additional features that may impact laptop prices, such as Processor, GPU, and other relevant attributes. Optimize feature selection to improve model performance.
5. User Authentication: Implement user authentication mechanisms for secure access to the application. Include features for user sign-up, login, and potentially an admin panel for managing user information.
6. Testing and Validation: Conduct extensive testing to validate the accuracy and reliability of machine learning models. Implement test cases and use realistic data scenarios to evaluate the system's performance.
7. Future Enhancements: Identify areas for future improvement, such as incorporating additional features, exploring alternative data sources, and optimizing model selection and performance.

1.3 Motivation of the Project:

The motivation behind developing the “Laptop Price Prediction System” arises from the increasing complexity and variability in laptop pricing across different markets and platforms. With the rapid advancement in technology, laptops are being released with a wide variety of configurations, including differences in processor type, RAM, storage, display quality, and graphics capabilities. These variations make it difficult for consumers to determine whether a particular laptop is priced fairly.

From a buyer’s perspective, there is always uncertainty regarding whether the price being offered is justified based on the specifications. Similarly, sellers and resellers often face challenges in determining competitive pricing that maximizes profit while remaining attractive to customers. This lack of transparency in pricing can lead to inefficient market decisions.

With the rise of data-driven decision-making and machine learning, it has become possible to analyze historical data and identify patterns that influence laptop pricing. This project is motivated by the need to leverage these technologies to create a system that can assist users in predicting laptop prices accurately.

Additionally, this project aligns with the growing demand for intelligent systems that simplify complex decision-making processes. By combining data analysis, machine learning, and a user-friendly interface developed using Python-based tools, the system aims to provide practical value to end users.

The project also serves as a learning opportunity to apply theoretical concepts of machine learning, data preprocessing, and model evaluation to a real-world problem, thereby enhancing both technical and analytical skills.

1.4 Real-World Problem Statement (Expanded Explanation):

In today's digital marketplace, laptop pricing is influenced by multiple dynamic factors such as brand reputation, hardware specifications, market demand, and technological trends. E-commerce platforms like Amazon and Flipkart list thousands of laptops with varying configurations, making it difficult for users to compare and evaluate prices effectively.

One of the key challenges is the absence of a standardized pricing mechanism. Two laptops with similar specifications may have significantly different prices due to branding, marketing strategies, or seller margins. This inconsistency creates confusion among buyers and leads to inefficient purchasing decisions.

Moreover, second-hand or refurbished laptop markets face even greater challenges due to the lack of proper valuation models. Sellers often rely on guesswork or market observation, which may not reflect the true value of the device.

This project addresses these real-world challenges by introducing a machine learning-based solution that predicts laptop prices based on historical data and key features. The system reduces dependency on manual estimation and provides a more data-driven and objective approach to pricing.

By doing so, the project contributes toward improving transparency, efficiency, and decision-making in the laptop marketplace.

1.5 Applications of the Project:

The Laptop Price Prediction System has a wide range of practical applications across different domains:

1. E-commerce Platforms: Online marketplaces can integrate such models to provide price recommendations and improve user trust.

2. Retailers and Resellers: Sellers can use the system to determine optimal pricing strategies for both new and refurbished laptops.

3. Consumers: Buyers can use the system to evaluate whether a laptop is reasonably priced before making a purchase decision.

4. Market Analysts: Analysts can use prediction insights to study pricing trends and consumer behavior in the electronics market.

5. Educational Purpose: The project can be used as a case study for understanding the application of machine learning in real-world scenarios.

6. Startups and Tech Platforms: New businesses dealing with electronics resale or comparison websites can use this system as a core feature.

Overall, the system provides value by enabling informed decision-making and reducing uncertainty in pricing.

1.6 Limitations of the Current System:

Despite its effectiveness, the current system has certain limitations:

1. Dataset Dependency: The accuracy of predictions depends heavily on the quality and size of the dataset used for training.

2. Static Data: The model is trained on historical data and does not account for real-time market fluctuations.

3. Limited Features: Some external factors such as brand perception, seasonal demand, and discounts are not included.

4. Generalization Issues: The model may not perform equally well for completely new or rare laptop configurations.

5. Deployment Constraints: The current deployment using a web interface may have limitations in scalability and performance under high traffic.

These limitations highlight areas for future improvement and enhancement of the system.

1.7 Overview of Technologies Used:

The project utilizes a combination of modern tools and technologies for development, implementation, and deployment:

* Programming Language: Python is used for data processing, model building, and application development.
* Data Analysis Libraries: Pandas and NumPy are used for data manipulation and preprocessing.
* Machine Learning Libraries: Scikit-learn is used for building and evaluating predictive models.
* Visualization Tools: Matplotlib and Seaborn are used for data visualization.
* Web Framework: Streamlit is used to develop an interactive web interface for users to input laptop specifications and get predictions.
* Deployment Platform: The application is deployed using Render, enabling online access to the system.

These technologies together enable the development of a complete end-to-end machine learning application.

CHAPTER 2: Theoretical Background & Definition of Problem

2.1 Theoretical Background:

The theoretical background of the "Laptop Price Prediction" project involves several key concepts in data science and machine learning. Here's a breakdown of the theoretical foundations:

1. Data Science Concepts:
    1. Data Cleaning and Preprocessing: Before applying machine learning algorithms, it's essential to clean and preprocess the dataset. This involves handling missing values and outliers and ensuring the data is in a suitable format.
    2. Exploratory Data Analysis (EDA): EDA is crucial for understanding the characteristics of the dataset. It involves generating summary statistics and visualizations and identifying patterns or trends in the data.
2. Machine Learning Principles:
    1. Supervised Learning: Our project involves supervised learning, where the model is trained on a labeled dataset to make predictions. In this case, predicting laptop prices is based on various features.
    2. Regression models, such as linear regression and random forest regression, are used for predicting continuous variables, like laptop prices.
    3. Classification Metrics: In addition to regression metrics, classification metrics like accuracy are employed. Accuracy provides an overall measure of correct predictions. However, considering the imbalance in classes, the confusion matrix and classification report are crucial for a more nuanced evaluation.

3. Feature Engineering:
    - Feature Selection and Transformation: Feature engineering is the process of selecting relevant features and transforming them to improve the model's performance. It's crucial for building accurate predictive models.

4. Model Evaluation Metrics:
    - Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE): These metrics are used to evaluate the performance of regression models. They quantify the difference between predicted and actual values.
5. Data Visualization:
    - Graphs and Charts: Data visualization tools, such as Matplotlib and Seaborn, are employed to create visual representations of data patterns, aiding in better understanding and communication.
6. User Authentication and Security:
    - User Authentication: Ensuring secure user authentication is crucial for protecting user data and maintaining the integrity of the application.
7. Overfitting and Underfitting:
    - Model Generalization: Understanding the balance between overfitting and underfitting is essential. Overfitting occurs when the model is too complex, while underfitting happens when it is too simple.
8. Ethical Considerations:
    - Fairness and Bias: Being mindful of potential biases in the dataset and ensuring that the model's predictions are fair and unbiased.

2.2 Introduction to Machine Learning:

- Machine Learning (ML) is a subset of Artificial Intelligence (AI) that enables systems to learn from data and improve their performance without being explicitly programmed. Instead of relying on predefined rules, machine learning algorithms identify patterns within data and use those patterns to make predictions or decisions.

- In the context of this project, machine learning is used to predict laptop prices based on historical data and various hardware specifications. This approach eliminates the need for manual estimation and provides a more data-driven and accurate pricing mechanism.

- Machine learning is widely used in various domains such as healthcare, finance, e-commerce, and marketing. In e-commerce, ML plays a crucial role in recommendation systems, demand forecasting, and price prediction.

2.3 Types of Machine Learning:

Machine learning can be broadly categorized into the following types:

1. Supervised Learning:

- Supervised learning involves training a model on a labeled dataset, where both input features and output values are known. The model learns the relationship between inputs and outputs and uses this knowledge to predict outcomes for new data.

- In this project, supervised learning is used because the dataset contains input features (like RAM, CPU, etc.) and corresponding output (price).

2. Unsupervised Learning:

- Unsupervised learning deals with unlabeled data. The model tries to identify patterns, clusters, or relationships within the data without any predefined output.

3. Reinforcement Learning:

- Reinforcement learning involves training an agent to make decisions by rewarding correct actions and penalizing incorrect ones.

Note: For this project, supervised learning (regression) is used.

2.4 Regression in Machine Learning:

- Regression is a type of supervised learning used to predict continuous numerical values. In this project, the target variable (price) is continuous, making regression the appropriate technique.

- The goal of regression is to find a mathematical relationship between independent variables (features) and the dependent variable (price).

2.5 Algorithms Used in the Project:

2.5.1 Linear Regression

- Linear Regression is one of the simplest and most widely used machine learning algorithms. It assumes a linear relationship between input variables and output.

Mathematical Representation:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \epsilon$$

Where:

- Y: Dependent variable (outcome).
- β_0: Intercept (value of Y when all $X_i = 0$).
- $\beta_1, \beta_2, \ldots, \beta_n$: Coefficients of the independent variables ($X_1, X_2, \ldots, X_n$), representing their impact on Y.
- $X_1, X_2, \ldots, X_n$: Independent variables (predictors).
- ϵ: Error term (accounts for randomness or factors not included in the model).

Advantages:

1. Simple and easy to interpret
2. Fast computation

Disadvantages:

1. Cannot capture complex relationships
2. Sensitive to outliers

2.5.2 Decision Tree Regression

- A Decision Tree is a tree-like model used for decision-making. It splits the dataset into smaller subsets based on feature values.

Working:

I. Select the best feature.

II. Split data into branches.

III. Continue until stopping condition.

Advantages:

1. Easy to understand.

2. Handles non-linear data.

Disadvantages:

1. Prone to overfitting.

2.5.3 Random Forest Regression (Best Model)

- Random Forest is an ensemble learning method that combines multiple decision trees to improve accuracy.

  Working:

  I. Multiple trees are created using random subsets of data

  II. Each tree makes a prediction

  III. Final output = average of all predictions

  Advantages:

  1. High accuracy

  2. Reduces overfitting

  3. Works well with large datasets

  Why used in this project:

  - Random Forest gave the highest R^2 score, making it the best-performing model.

2.6 Feature Engineering:

- Feature Engineering is the process of selecting, modifying, or creating new features to improve model performance.

  Examples from this project:

  1. Converting RAM from string to integer

  2. Extracting CPU brand

  3. Creating PPI (Pixels Per Inch)

4. Splitting storage into SSD and HDD

Feature engineering improves accuracy by making data more meaningful for the model.

2.7 Data Preprocessing Techniques:

Before applying machine learning models, data must be cleaned and prepared.

Steps involved:

I. Handling Missing Values:

Missing data can lead to incorrect predictions. It is handled by:

i. Removing rows

ii. Filling values (mean/median)

II. Removing Outliers:

Outliers are extreme values that can distort results.

III. Encoding Categorical Data:

Machine learning models require numerical input, so categorical variables are converted using:

i. One-Hot Encoding

IV. Feature Scaling:

Ensures all features are on the same scale.

2.8 Model Evaluation Metrics:

To measure model performance, the following metrics are used:

1. Mean Absolute Error (MAE)

$$MAE = \frac{1}{n}\sum |y_i - \hat{y}_i|$$

- Measures average error
- Lower value = better model

2. Mean Squared Error (MSE)

$$MSE = \frac{1}{n}\sum (y_i - \hat{y}_i)^2$$

- Penalizes larger errors

3. Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

- Easier to interpret than MSE

4. R^2 Score (Coefficient of Determination)

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

- Measures how well the model fits data
- Value closer to 1 = better

2.9 Overfitting and Underfitting:

Overfitting-

i. Occurs when the model learns too much from training data, including noise.

ii. High accuracy on training data

iii. Poor performance on new data

Underfitting-

i. Occurs when the model is too simple to capture patterns.

ii. Poor accuracy on both training and testing data.

2.10 Bias-Variance Tradeoff:

* Bias: Error due to simplicity of model

* Variance: Error due to complexity

A good model balances both.

2.11 Data Visualization:

Visualization helps understand patterns in data.

Tools used:

1. Matplotlib

2. Seaborn

Examples:

1. Bar charts

2. Distribution plots

3. Scatter plots

2.12 Ethical Considerations:

Machine learning models must be fair and unbiased.

Issues:

1. Biased dataset

2. Incorrect predictions

Solutions:

1. Use diverse dataset

2. Validate model properly

2.13 Definition of Problem:

The problem at the core of the "Laptop Price Prediction" project is the inherent challenge in the buying and selling process of automobiles. This challenge revolves around the determination of a price point that is mutually agreeable to both the seller and the buyer. Striking this delicate balance is crucial for a successful transaction, where the buyer aims to secure a fair deal without overpaying, and the seller seeks to set a price that reflects the true value of the vehicle.

To overcome this intricate problem, the project addresses the need for a reliable and data-driven approach. It utilizes a dataset comprising diverse Laptops from various manufacturers, each annotated with crucial attributes such as Processor, GPU, and other specifications. The objective is to employ machine learning models that can analyze these features and accurately predict Laptop prices.

In the process of tackling this challenge, the project takes a multifaceted approach. It involves comprehensive data analysis, including cleaning and visualization using graphs and charts, to understand underlying patterns and trends in the dataset. The application of machine learning models at the conclusion of the analysis phase adds a predictive layer, enabling the accurate estimation of Laptop prices.

CHAPTER 3: System Analysis

3.1 System Analysis:

System analysis is a crucial phase in the development of any information system or software application. It involves a detailed study of the various components of a system, their interrelationships, and their interactions with the environment. The goal of system analysis is to understand the requirements of the system, identify areas for improvement, and propose solutions that align with the objectives of the organization or project.

The key components of system analysis:

1. Feasibility Study:

A feasibility study is an assessment of the practicality, viability, and potential success of a proposed project or system. It aims to determine whether the project is technically, economically, and operationally feasible within the constraints and requirements defined. The study involves an in-depth analysis of various aspects, including technical capabilities, financial considerations, and operational requirements.

Here are the key dimensions typically considered in a feasibility study:

a) Technical Feasibility-

Technical feasibility evaluates whether the system can be developed using available technology.

Key Points:

1. Python ecosystem (Pandas, NumPy, Scikit-learn) supports full ML pipeline.
2. Streamlit enables easy web app creation.
3. Deployment on Streamlit Cloud / Render is simple and cost-effective.

Challenges:

1. Handling large datasets
2. Model optimization
3. Feature extraction from complex strings

Conclusion:

Technically feasible with modern tools.

b) Economic Feasibility-

- Objective: To determine the financial viability of the project.

Key Considerations:

- Estimation of costs: The main expenses include the development of the machine learning model, software development, and maintenance. These costs are manageable and within budget.
- Potential returns on investment (ROI): The project's potential for ROI is based on improved decision-making in the laptop market, benefiting buyers and sellers.
- Cost-benefit analysis: A cost-benefit analysis indicates that the benefits of the project outweigh the costs.
- Budget constraints: The project is within budget constraints.

c) Operational Feasibility-

- Objective: To assess whether the project can be smoothly integrated into existing operations and effectively serve its purpose.

Key Considerations:

- Alignment with operational requirements: The project aligns with the goal of providing accurate Laptop price predictions, which is essential for Laptop buyers and sellers.
- Impact on day-to-day processes and workflow: The end-user interface is user-friendly, requiring minimal training for users.

- User acceptance and willingness to adopt the new system: The project aims to improve the decision-making process in the Laptop market, and users are likely to adopt it for its benefits.
- Identification of operational risks and challenges: Any operational challenges are expected to be minimal and manageable.

d) Other Feasibility Dimensions—

- Objective: To explore any additional factors that might impact the project's success.

Key Considerations:

- Legal and regulatory compliance: The project should comply with data privacy and any relevant regulations.
- Environmental impact: The project has no significant environmental impact.
- Social and political factors: The project's social and political impact is expected to be positive, aiding Laptop buyers and sellers in making informed decisions.

2. Analysis Methodology:

The analysis methodology is a critical phase in the development of the Laptop Price Prediction project, as it involves processing and interpreting the data, selecting the appropriate machine learning models, and assessing their performance.

The methodology is divided into several key steps:

a) Data Collection and Pre-processing:

Objective: Gather relevant data and prepare it for analysis.

Key Activities:

- Data Collection: Collect data related to Laptop features, including manufacturer, model, year, category, interior, Display Type, GPU, Processor, Storage, and price.
- Data Cleaning: Clean the dataset by handling missing values, outliers, and inconsistent data.
- Data Transformation: Convert categorical variables into numerical format, perform feature scaling if necessary, and prepare the dataset for analysis.

b) Data Analysis and Visualization:

- Objective: Understand the dataset and extract insights through data analysis and visualization.
- Key Activities:

- Exploratory Data Analysis (EDA): Perform EDA to uncover patterns, correlations, and trends in the data.
- Data Visualization: Create graphs, charts, and visualizations to represent key findings, making complex information more accessible.

c) Model Selection and Training:

- Objective: Choose the most suitable machine learning models for Laptop price prediction and train them.

Key Activities:

- Model Selection: Evaluate various regression models such as Random Forest, Linear Regression and choose the most appropriate model for this regression problem.
- Data Splitting: Divide the dataset into training and testing sets to train and evaluate the selected model.
- Model Training: Train the chosen model on the training data.

d) Model Evaluation and Tuning:

Objective: Assess model performance, make necessary adjustments, and optimize the chosen model. Key Activities:

- Model Evaluation: Measure model performance using evaluation metrics like Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE).
- Hyperparameter Tuning: Optimize the model by tuning hyperparameters for better predictive accuracy.
- Cross-Validation: Apply cross-validation techniques to validate the model's generalization performance.

3. Choice of platform:

- The choice of platforms in a software development project involves decisions regarding the software and hardware environments on which the system will be developed, deployed, and run. Here's a breakdown of the key components:

(e) Software used:

In this project, various software tools and frameworks are employed to facilitate data analysis, model development, deployment, and user interaction.

Key Software Components:

1. Python: Python serves as the primary programming language for data analysis, machine learning model development, and web application deployment. It offers a wide range of libraries and frameworks, making it a versatile choice.
2. Machine Learning Libraries: Scikit-Learn, XGBoost, and other machine learning libraries are utilized for building and training machine learning regression models to predict Laptop prices.

3. Pandas and NumPy: These libraries are used for data manipulation, pre-processing, and numerical operations.
4. Jupyter Notebook: Jupyter Notebook is employed for data analysis and model prototyping. It offers an interactive environment for code development and visualization.
5. Pickle: Pickle is used to serialize and save machine learning models.
6. Streamlit: A Python-based open-source framework used to build and deploy interactive web applications for machine learning models quickly and efficiently.
7. Render: A cloud-based deployment platform that allows developers to host and run web applications (including Streamlit apps) easily with free and scalable infrastructure.
8. Other Python Packages: Various other Python packages are used for tasks like data visualization (Matplotlib, Seaborn) and data encoding.

(f) Hardware (H/W) Used:

The hardware resources selected for the project need to provide the necessary computational power for data analysis, model training, and web application deployment.

Key Hardware Components:

1. Computer/Server: A personal computer or server with sufficient computing capabilities is required. The hardware should meet the demands of data analysis and machine learning model training.
2. Processor (CPU): A multi-core processor, such as Intel Core i3 or AMD Ryzen, is beneficial for parallel processing and faster model training.
3. Memory (RAM): A minimum of 8GB RAM is recommended to handle large datasets and machine learning tasks efficiently.
4. Storage: Adequate storage space, preferably an SSD (Solid State Drive) for faster data access, is essential to store datasets, models, and application files.
5. Operating System: The project can be developed on various operating systems, including Windows, macOS, or Linux, depending on your preference and requirements.

6. Internet Connectivity: Internet connectivity is essential for data collection, software updates, and potential cloud-based deployment.

1. Machine Learning Model:

- Algorithm: Random Forest Regression (or any chosen algorithm).
- Training: Model trained on a dataset with Laptop features and prices.
- Serialization: Model saved for future use.

2. Data Storage:

- Dataset Storage: Raw dataset with Laptop features and prices.
- Model Storage: Serialized machine learning model.

3. External Libraries and Tools:

- Pandas: Used for data manipulation and cleaning.
- Scikit-Learn: Employed for building and training machine learning models.

CHAPTER 4: System Planning

System Planning:

1. Objective Definition:
    i) Objective: Develop a machine learning-based system that accurately predicts the market price of Laptops based on key attributes.

2. Project Scope:
    i) Inclusions: Data collection, cleaning, analysis, machine learning model development
    ii) Exclusions: Direct real-time market integration, extensive historical price tracking.

3. Resource Identification:
    i) Human Resources: Data scientists, developers, UI/UX designers, project manager.
    ii) Technology: Python, machine learning libraries (Scikit-learn).
    iii) Infrastructure: Cloud platform for hosting and scalability.

4. Feasibility Analysis:
    i) Technical Feasibility: Python's extensive libraries support machine learning.
    ii) Economic Feasibility: Cost-effective using open-source tools and cloud resources.
    iii) Other Feasibility Dimensions: Compatibility with various devices and browsers.

5. Risk Assessment:
    i) Technical Risks: Model accuracy, potential bias in predictions.
    ii) Operational Risks: Deployment issues, user adaptation.
    iii) Project Risks: Delays due to unforeseen challenges.
    iv) Stakeholder Updates: Bi-weekly reports on progress.
    v) User Feedback Sessions: Periodic sessions for user input.

6. Gantt Chart:

Phase	Duration
Data Collection	2 weeks
EDA	3 weeks
Model Training	4 weeks
Deployment	2 weeks

Benefits of System Planning:

- Clear Roadmap: Provides a clear plan from data collection to deployment.
- Resource Allocation: Ensures efficient use of resources.
- Risk Mitigation: Identifies and addresses potential risks.

CHAPTER 5: Methodology adopted, System Implementation & Details of Hardware & Software Used, System Maintenance & Evaluation

5.1 Methodology adopted:

The methodology adopted in our project typically includes various stages, from conceptualization to implementation. Here's a general breakdown of the methodology:

1) Problem Definition: Clearly define the problem you aim to solve, which, in your case, is predicting Laptop prices accurately.
2) Literature Review: Research existing solutions, methodologies, and technologies related to Laptop price prediction and machine learning.
3) Data Collection: Gather a comprehensive dataset containing information on Laptops, including features like Processor, GPU, and wheels, along with their corresponding prices.
4) Data Pre-processing: Clean and prepare the dataset for analysis, addressing missing values and outliers and ensuring data quality.
5) Exploratory Data Analysis (EDA): Perform EDA to understand the characteristics of the dataset, identify patterns, correlations, and gain insights into the relationships between different features.
6) Feature Engineering: Select relevant features and create new ones that might enhance the predictive power of your model.
7) Model Selection: Choose suitable machine learning algorithms for your problem. Considering the nature of your problem (Laptop price prediction), regression models like Random Forest, Linear Regression, etc., are commonly used.
8) Model Development: Train and optimize your selected machine learning models using the prepared dataset.
9) Evaluation Metrics: Employ metrics like Mean Absolute Error (MAE), Mean Squared Error (MSE), and R-squared to assess the performance of your models.

5.2 Details of Hardware & Software used System Maintenance & Evaluation:

Software (S/W) Used:

In this project, various software tools and frameworks are employed to facilitate data analysis, model development, deployment, and user interaction.

Key Software Components:

1. Python: Python serves as the primary programming language for data analysis, machine learning model development, and web application deployment. It offers a wide range of libraries and frameworks, making it a versatile choice.
2. Machine Learning Libraries: Scikit-Learn and other machine learning libraries are utilized for building and training machine learning regression models to predict Laptop prices.
3. Pandas and NumPy: These libraries are used for data manipulation, pre-processing, and numerical operations.
4. Jupyter Notebook: Jupyter Notebook is employed for data analysis and model prototyping. It offers an interactive environment for code development and visualization.
5. Pickle: Pickle is used to serialize and save machine learning models.
6. Stream lit: A Python-based open-source framework used to build and deploy interactive web applications for machine learning models quickly and efficiently.
7. Render: A cloud-based deployment platform that allows developers to host and run web applications (including Streamlit apps) easily with free and scalable infrastructure.
8. Other Python Packages: Various other Python packages are used for tasks like data visualization (Matplotlib, Seaborn) and data encoding.

Hardware (H/W) Used:

The hardware resources selected for the project need to provide the necessary computational power for data analysis, model training, and web application deployment.

Key Hardware Components:

1. Computer/Server: A personal computer or server with sufficient computing capabilities is required. The hardware should meet the demands of data analysis and machine learning model training.

2. Processor (CPU): A multi-core processor, such as Intel Core i3 or AMD Ryzen, is beneficial for parallel processing and faster model training.

3. Memory (RAM): A minimum of 8GB RAM is recommended to handle large datasets and machine learning tasks efficiently.

4. Storage: Adequate storage space, preferably an SSD (Solid State Drive) for faster data access, is essential to store datasets, models, and application files.

5. Operating System: The project can be developed on various operating systems, including Windows, macOS, or Linux, depending on your preference and requirements.

6. Internet Connectivity: Internet connectivity is essential for data collection, software updates, and potential cloud-based deployment.

5.3 System maintenance & evaluation

System maintenance involves the ongoing process of managing, updating, and enhancing the system after it has been deployed. It ensures that the system continues to meet its objectives and remains effective. Key aspects of system maintenance include:

- Bug Fixes and Updates:

  Addressing any bugs or issues that arise after deployment and providing updates to improve system performance.

- Security Updates:

  Regularly updating the system to address security vulnerabilities and protect against potential threats.

- Performance Optimization:

  Identifying and implementing improvements to optimize system performance, such as addressing bottlenecks or enhancing efficiency.

- Database Maintenance:

  Managing and maintaining the integrity of the system's databases, including data backup and recovery procedures.

System Evaluation:

System evaluation involves assessing the performance, effectiveness, and overall success of the system. This process helps in identifying areas for improvement and ensuring that the system aligns with its intended goals. Key aspects of system evaluation include:

- Performance Metrics:

  Defining and measuring performance metrics to evaluate how well the system is meeting its objectives.

- Scalability:

  Assessing the system's ability to scale and handle increased load or user demand over time.

- Reliability and Availability:

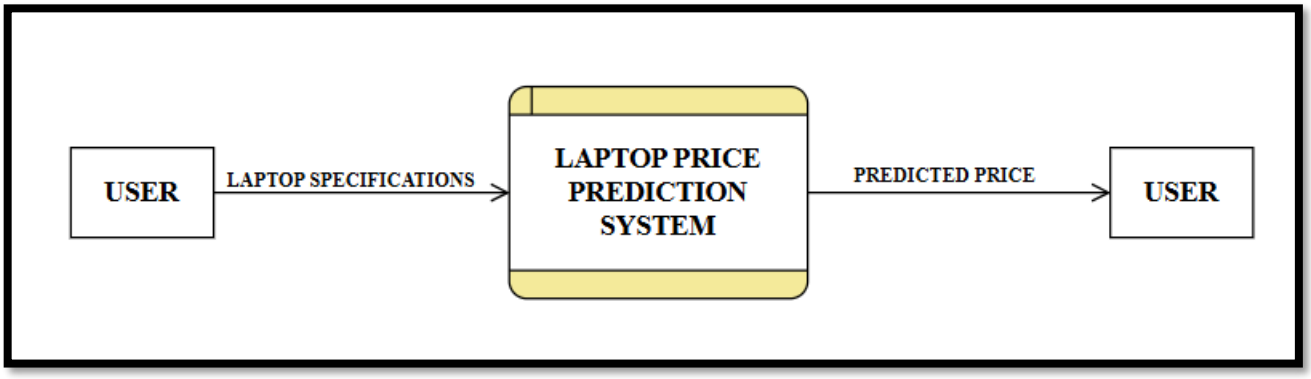
  Evaluating the reliability and availability of the system to ensure it meets the required standards.

- Security Assessment:

  Regularly assessing the system's security measures to identify and address potential vulnerabilities.

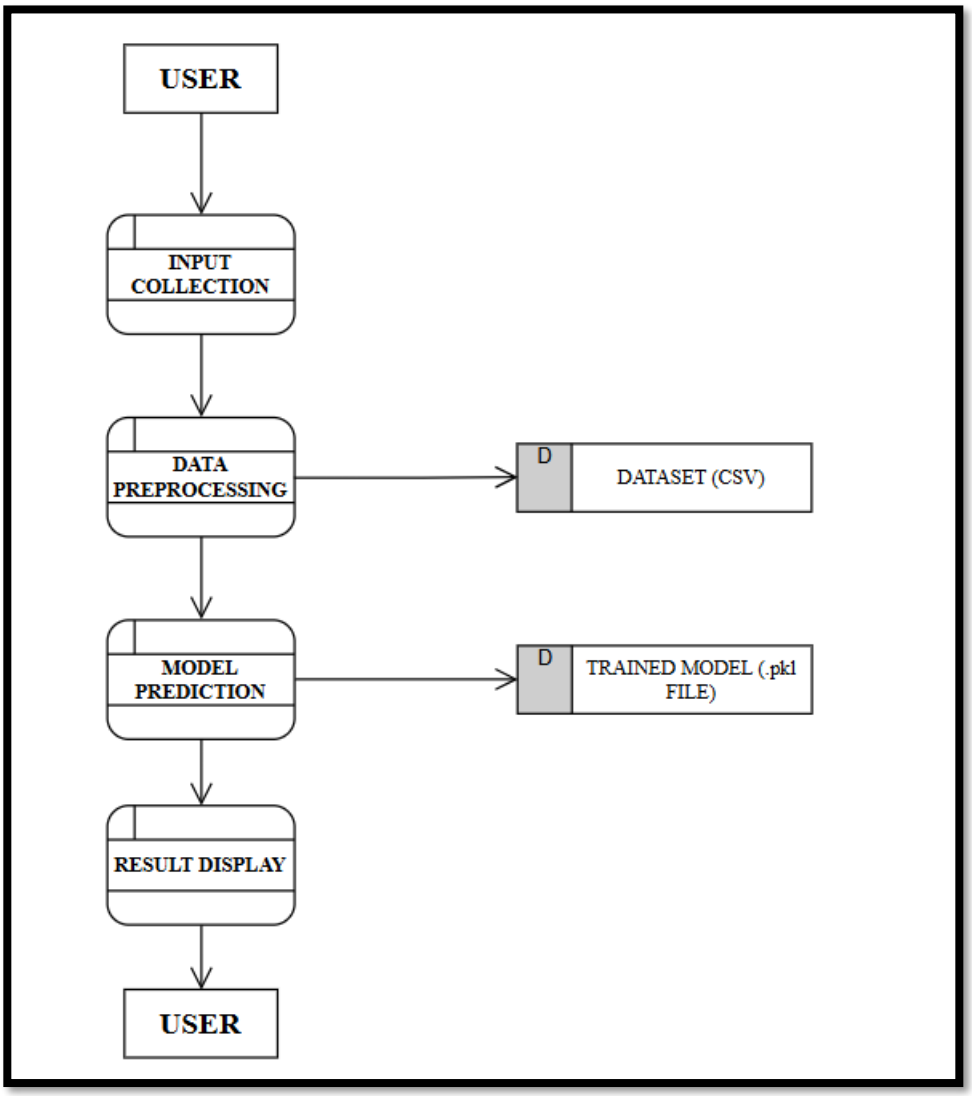
5.4 Data Flow Diagram (DFD):

Level 0 DFD:



The Level 0 Data Flow Diagram represents the overall functioning of the Laptop Price Prediction System as a single process. The user provides laptop specifications such as brand, RAM, processor, and storage as input to the system. The system processes this input using a trained machine learning model and generates the predicted price as output. This diagram provides a high-level overview of the system without showing internal processes.

Level 1 DFD:



The Level 1 Data Flow Diagram provides a detailed view of the internal processes of the system. Initially, the user inputs laptop specifications through the input collection module. The data is then passed to the preprocessing stage, where it is cleaned and transformed into a suitable format. The processed data is then sent to the model prediction module, where a trained machine learning model (stored as a .pkl file) predicts the laptop price. Finally, the result is displayed to the user. The dataset is used during the preprocessing and model training stages.

CHAPTER 6: Detailed Life Cycle of the Project

6.1 Dataset Description:

"Dataset description" refers to providing a comprehensive overview of the dataset used in a project. This information is crucial for understanding the context, structure, and content of the data.

Dataset Name	Laptop Price
Dataset Link	https://github.com/Sagar-Gupta008/render-demo-lpp/blob/master/laptop_data.csv
Number of Rows & Columns	1302 rows × 12 columns
Prediction column of the dataset	Price

6.2 Process involved:

Processes Involved in Laptop Price Prediction Project:

1) Data Collection:

- Definition: Gathering a diverse dataset of Laptop information from various manufacturers, including features like Processor, GPU, and price.
- Activities: Web scraping, API integration, or sourcing data from reputable sources.

2) Data Cleaning:

- Definition: Pre-processing raw data to handle missing values, outliers, and inconsistencies.
- Activities: Imputation, outlier removal, Standardization and normalization.

3) Data Analysis:
    - Definition: Examining the dataset to derive meaningful insights, patterns, and correlations.
    - Activities: Exploratory Data Analysis (EDA), statistical analysis, and visualization.
4) Feature Engineering:
    - Definition: Creating new features or modifying existing ones to enhance model performance.
    - Activities: Creating interaction terms, handling categorical variables, and scaling features.
5) Model Development:
    - Definition: Building machine learning models to predict laptop prices based on input features.
    - Activities: Selecting algorithms, training models, hyperparameter tuning, and evaluating performance.
6) Model Evaluation:
    - Definition: Assessing the accuracy and reliability of the trained models.
    - Activities: Using metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and R-squared.

6.3 Coding:

- **Importing Libraries**

import pandas as
pdimport numpy
as np
df=pd.read_csv("laptop_data.csv")
pd.read_csv("Laptop_price_prediction.csv")
df.head()

	Unnamed: 0	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory	Gpu	OpSys	Weight	Price
0	0	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 2.3GHz	8GB	128GB SSD	Intel Iris Plus Graphics 640	macOS	1.37kg	71378.6832
1	1	Apple	Ultrabook	13.3	1440x900	Intel Core i5 1.8GHz	8GB	128GB Flash Storage	Intel HD Graphics 6000	macOS	1.34kg	47895.5232
2	2	HP	Notebook	15.6	Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8GB	256GB SSD	Intel HD Graphics 620	No OS	1.86kg	30636.0000
3	3	Apple	Ultrabook	15.4	IPS Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16GB	512GB SSD	AMD Radeon Pro 455	macOS	1.83kg	135195.3360
4	4	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 3.1GHz	8GB	256GB SSD	Intel Iris Plus Graphics 650	macOS	1.37kg	96095.8080

- Information

  df.shape

```
df.shape

(1303, 12)
```

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1303 entries, 0 to 1302
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Unnamed: 0        1303 non-null   int64
 1   Company           1303 non-null   object
 2   TypeName          1303 non-null   object
 3   Inches            1303 non-null   float64
 4   ScreenResolution  1303 non-null   object
 5   Cpu               1303 non-null   object
 6   Ram               1303 non-null   object
 7   Memory            1303 non-null   object
 8   Gpu               1303 non-null   object
 9   OpSys             1303 non-null   object
 10  Weight            1303 non-null   object
 11  Price             1303 non-null   float64
dtypes: float64(2), int64(1), object(9)
memory usage: 122.3+ KB
```

- Data Cleaning

df.duplicated().sum()

```
df.duplicated().sum()

0
```

df.isnull().sum()

```
Unnamed: 0          0
Company             0
TypeName            0
Inches              0
ScreenResolution    0
Cpu                 0
Ram                 0
Memory              0
Gpu                 0
OpSys               0
Weight              0
Price               0
dtype: int64
```

df.drop(columns=['Unnamed:'],inplace=True)

df.head()

	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory	Gpu	OpSys	Weight	Price
0	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 2.3GHz	8GB	128GB SSD	Intel Iris Plus Graphics 640	macOS	1.37kg	71378.6832
1	Apple	Ultrabook	13.3	1440x900	Intel Core i5 1.8GHz	8GB	128GB Flash Storage	Intel HD Graphics 6000	macOS	1.34kg	47895.5232
2	HP	Notebook	15.6	Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8GB	256GB SSD	Intel HD Graphics 620	No OS	1.86kg	30636.0000
3	Apple	Ultrabook	15.4	IPS Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16GB	512GB SSD	AMD Radeon Pro 455	macOS	1.83kg	135195.3360
4	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 3.1GHz	8GB	256GB SSD	Intel Iris Plus Graphics 650	macOS	1.37kg	96095.8080

 # Removing the word "GB" in the RAM column

```
df['Ram']=df['Ram'].str.replace('GB','')
```

 # Removing the word kg in Weight column

```
df['Weight']=df['Weight'].str.replace('kg','')
df.head()
```

	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory	Gpu	OpSys	Weight	Price
0	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 2.3GHz	8	128GB SSD	Intel Iris Plus Graphics 640	macOS	1.37	71378.6832
1	Apple	Ultrabook	13.3	1440x900	Intel Core i5 1.8GHz	8	128GB Flash Storage	Intel HD Graphics 6000	macOS	1.34	47895.5232
2	HP	Notebook	15.6	Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8	256GB SSD	Intel HD Graphics 620	No OS	1.86	30636.0000
3	Apple	Ultrabook	15.4	IPS Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16	512GB SSD	AMD Radeon Pro 455	macOS	1.83	135195.3360
4	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 3.1GHz	8	256GB SSD	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080

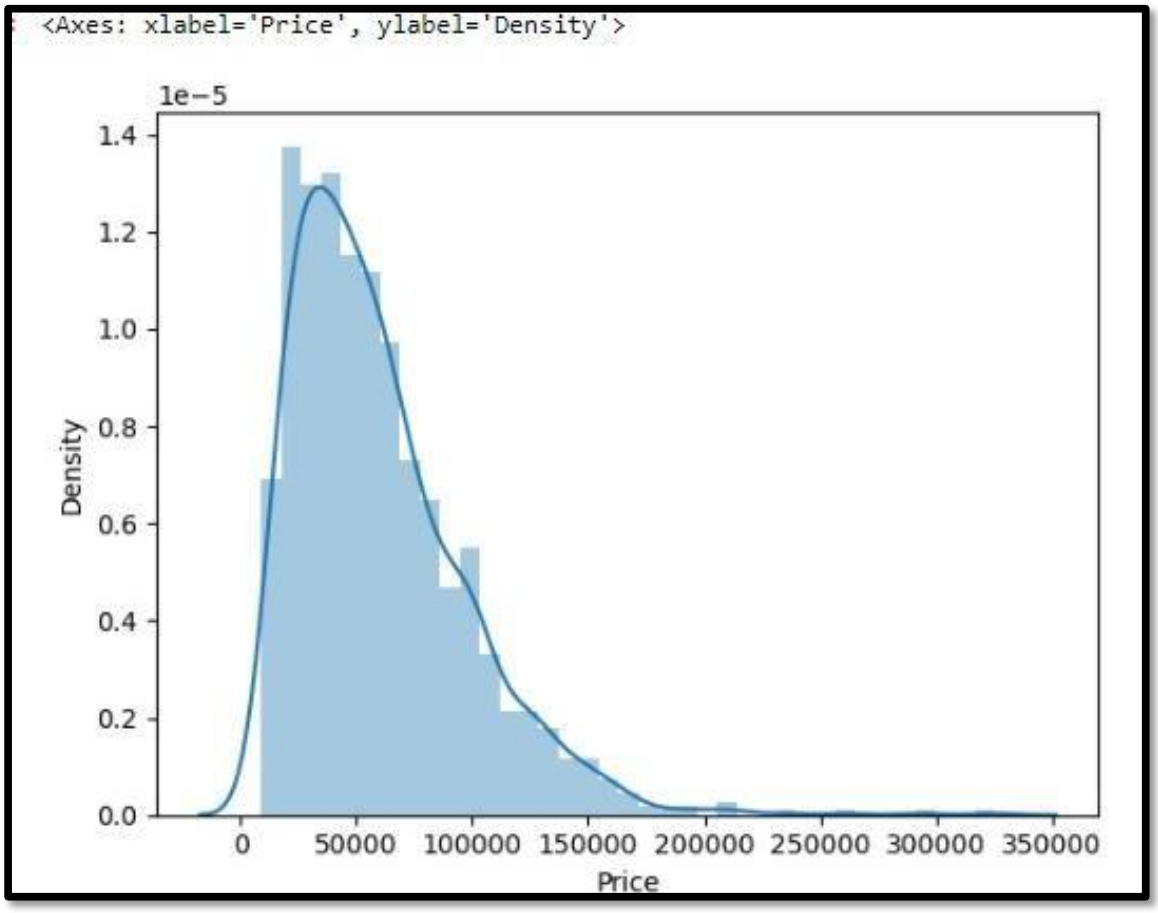
```
df['Ram']=df['Ram'].astype('int32')
df['Weight']=df['Weight'].astype('float32')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1303 entries, 0 to 1302
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Company           1303 non-null   object
 1   TypeName          1303 non-null   object
 2   Inches            1303 non-null   float64
 3   ScreenResolution  1303 non-null   object
 4   Cpu               1303 non-null   object
 5   Ram               1303 non-null   int32
 6   Memory            1303 non-null   object
 7   Gpu               1303 non-null   object
 8   OpSys             1303 non-null   object
 9   Weight            1303 non-null   float32
 10  Price             1303 non-null   float64
dtypes: float32(1), float64(2), int32(1), object(7)
memory usage: 101.9+ KB
```

- **Data Visualization**

  import seaborn as sns

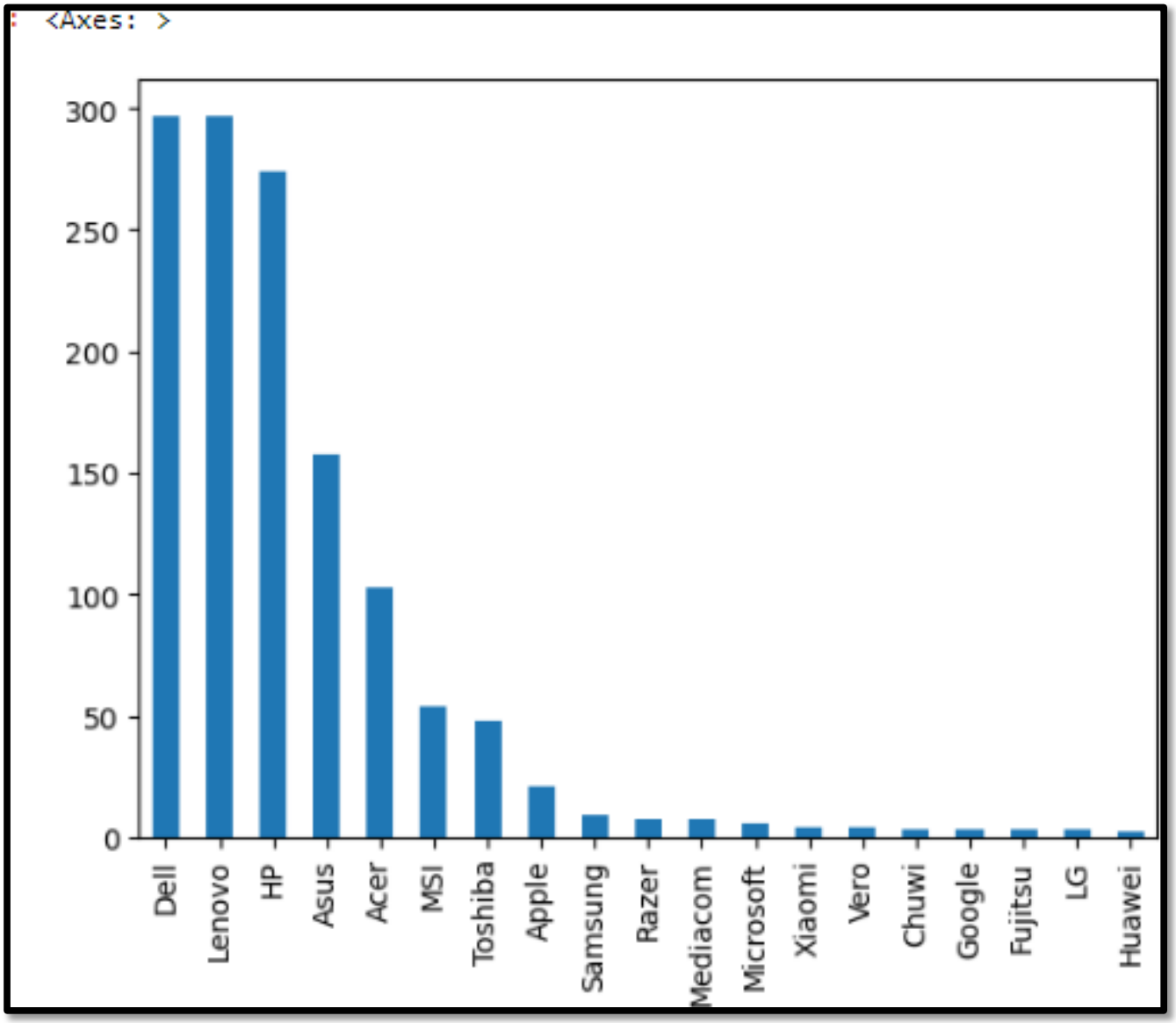
  sns.distplot(df['Price'])



From this we conclude that data is skewed, as there are more low -priced laptops than high -priced laptops.

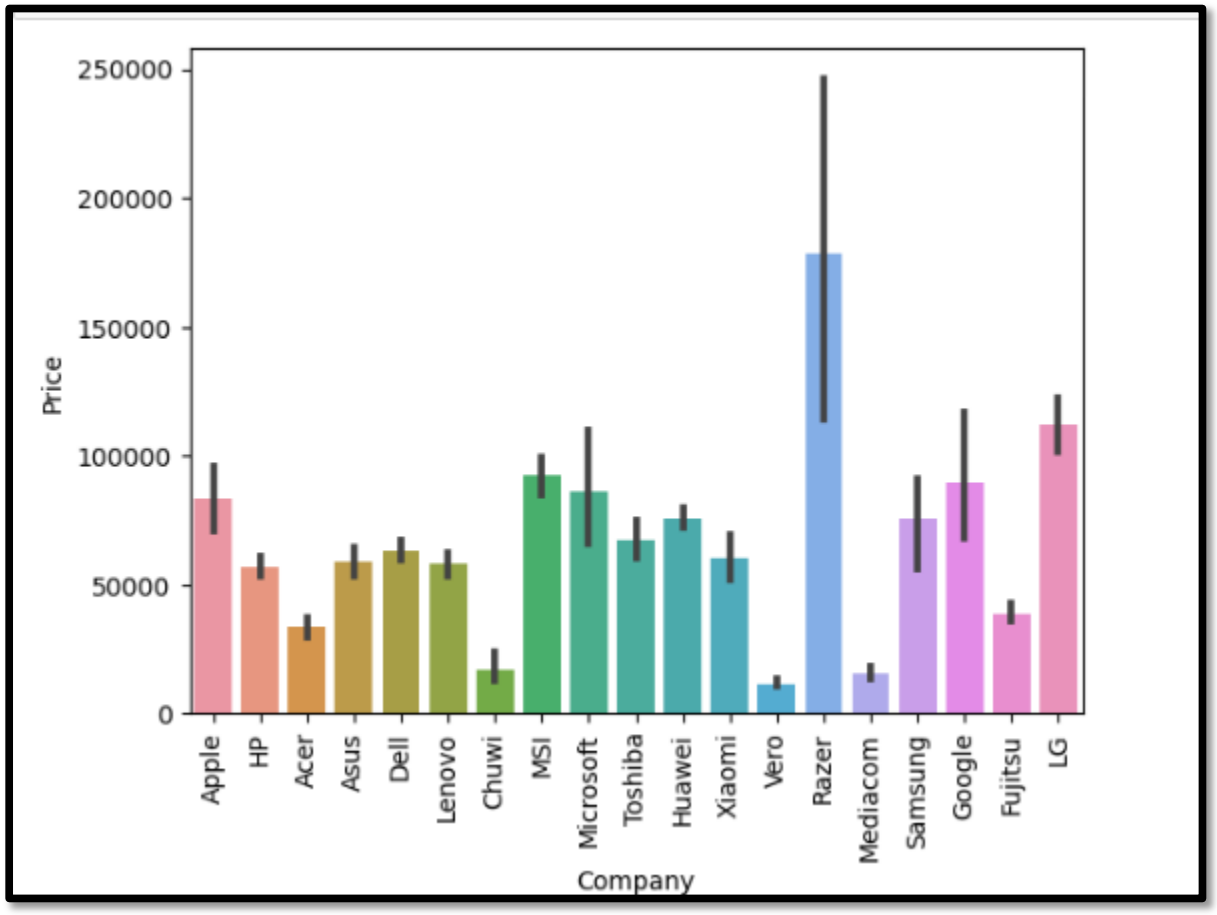
 # Checking number of laptops for each company present in the dataset

df['Company'].value_counts().plot(kind='bar')



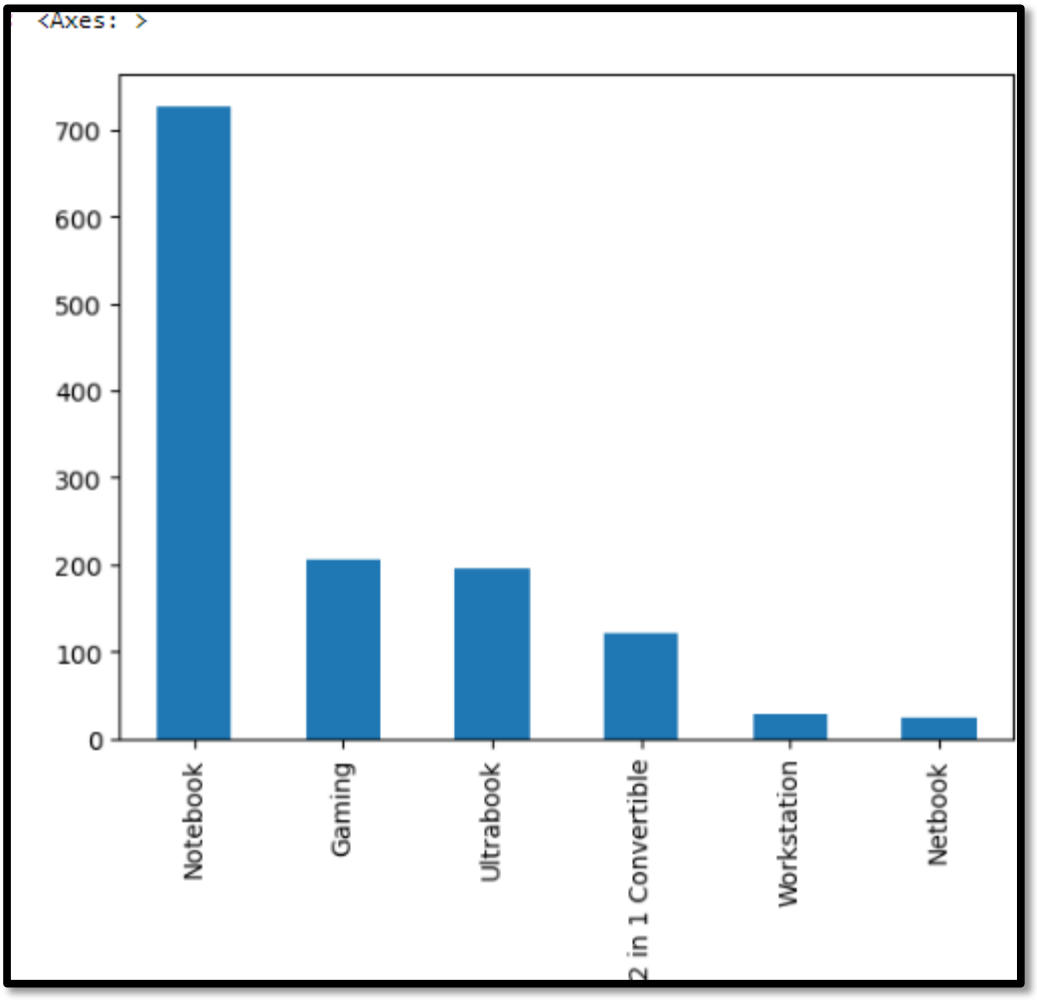
 # checking which company sells most and least expensive laptops

```
import matplotlib.pyplot as plt
sns.barplot(x=df['Company'],y=df['Price'])
plt.xticks(rotation='vertical')
plt.show()
```



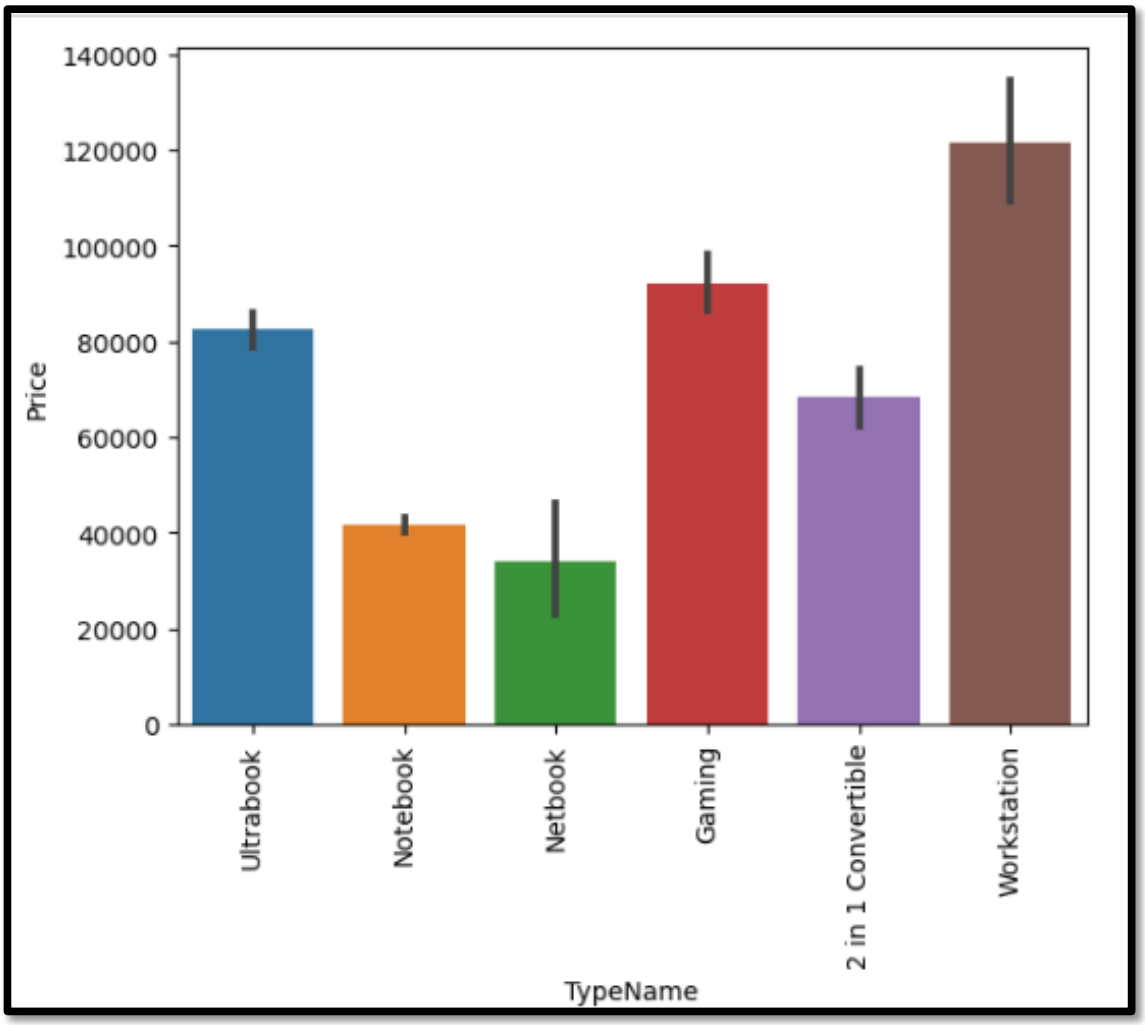
 # Checking different kinds of laptops that are present in the dataset

df['TypeName'].value_counts().plot(kind='bar')



 # Checking which type of laptop is most and least expensive

```
import matplotlib.pyplot as plt
sns.barplot(x=df['TypeName'],y=df['Price'])
plt.xticks(rotation='vertical')
plt.show()
```



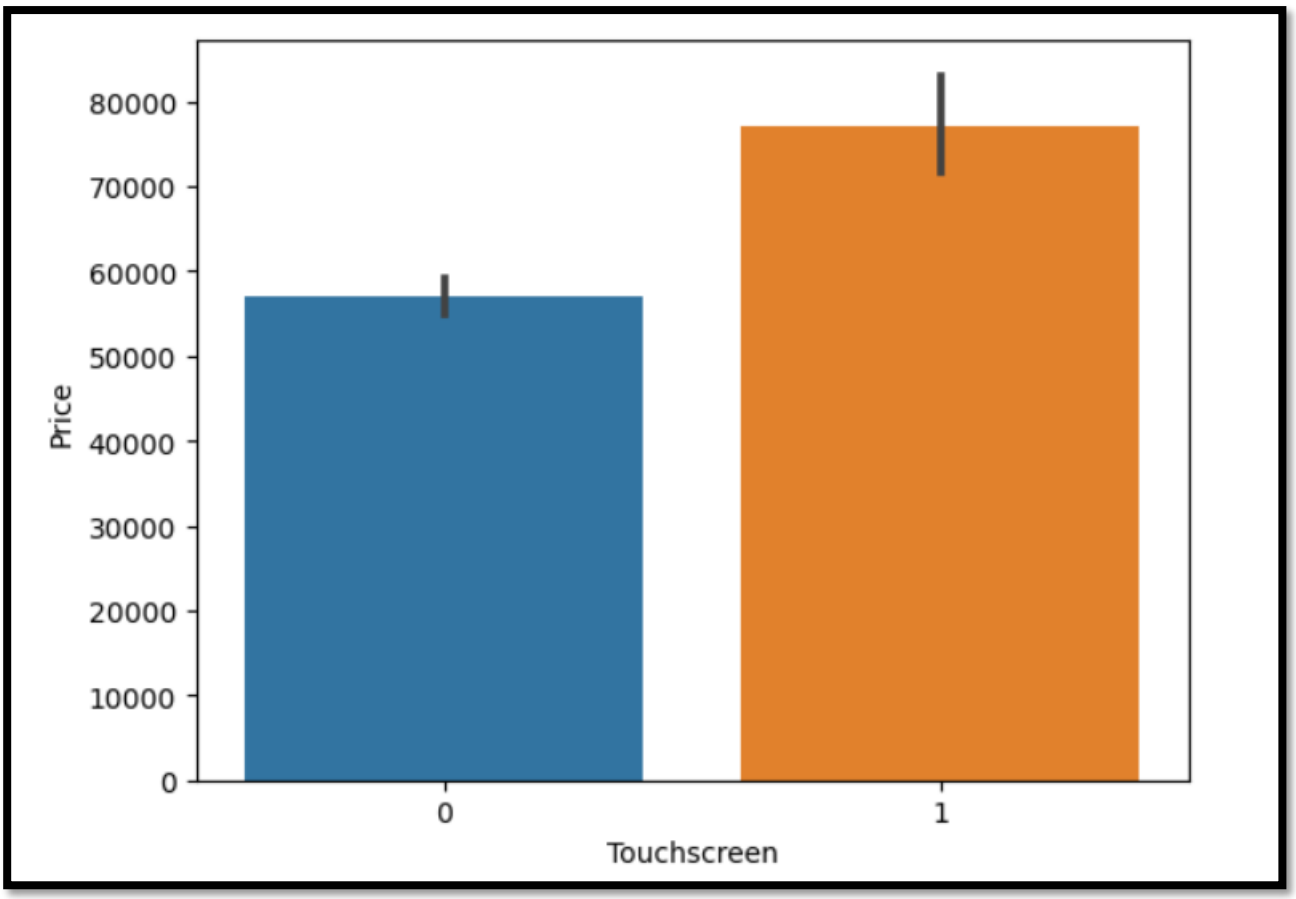
- Feature engineering on ScreenResolution column

 # making a new column, Touchscreen, for Checking if the laptop is a touchscreen or not

```
df['Touchscreen']=df['ScreenResolution'].apply(lambda x:1 if 'Touchscreen' in x else 0)
df.head()
```

	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory	Gpu	OpSys	Weight	Price	Touchscreen
0	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 2.3GHz	8	128GB SSD	Intel Iris Plus Graphics 640	macOS	1.37	71378.6832	0
1	Apple	Ultrabook	13.3	1440x900	Intel Core i5 1.8GHz	8	128GB Flash Storage	Intel HD Graphics 6000	macOS	1.34	47895.5232	0
2	HP	Notebook	15.6	Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8	256GB SSD	Intel HD Graphics 620	No OS	1.86	30636.0000	0
3	Apple	Ultrabook	15.4	IPS Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16	512GB SSD	AMD Radeon Pro 455	macOS	1.83	135195.3360	0
4	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 3.1GHz	8	256GB SSD	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080	0

```
sns.barplot(x=df['Touchscreen'],y=df['Price'])
```



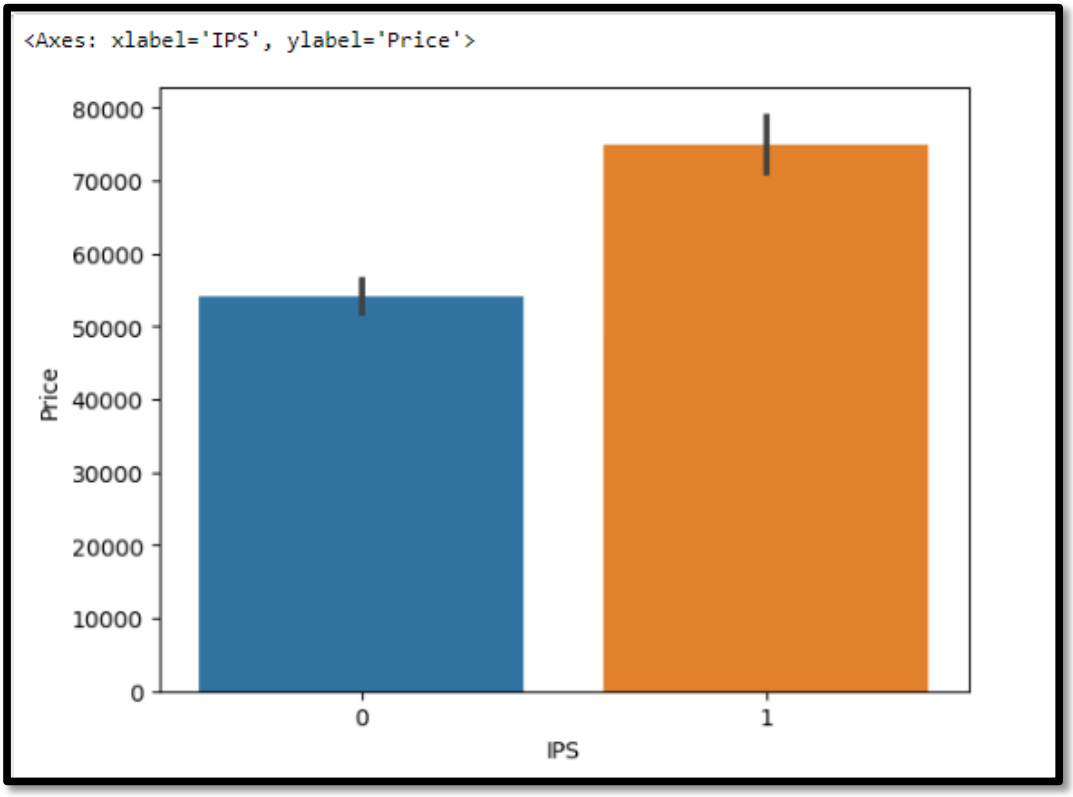
From here we conclude that touchscreen laptops are more expensive than laptops that are not touchscreens.

 # making a new column IPS for Checking if the laptop has IPS panel or not

```
df['IPS']=df['ScreenResolution'].apply(lambda x:1 if 'IPS' in x else 0)
df.sample(5)
```

	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory	Gpu	OpSys	Weight	Price	Touchscreen	IPS
278	Lenovo	Notebook	17.3	1600x900	Intel Core i3 6006U 2GHz	4	1TB HDD	Intel HD Graphics 520	Windows 10	2.80	26053.92	0	0
493	Acer	Notebook	15.6	1366x768	AMD A10-Series 9620P 2.5GHz	8	1TB HDD	AMD Radeon RX 540	Windows 10	2.20	30849.12	0	0
207	Dell	Ultrabook	13.3	IPS Panel 4K Ultra HD / Touchscreen 3840x2160	Intel Core i7 8550U 1.8GHz	8	256GB SSD	Intel UHD Graphics 620	Windows 10	1.21	103842.72	1	1
450	HP	Notebook	15.6	1366x768	Intel Core i5 7200U 2.5GHz	8	128GB SSD	Intel HD Graphics 620	Windows 10	1.91	31381.92	0	0
408	Lenovo	Notebook	15.6	Full HD 1920x1080	Intel Core i3 6006U 2GHz	4	500GB HDD	Intel HD Graphics 520	Windows 10	2.20	21791.52	0	0

```
sns.barplot(x=df['IPS'],y=df['Price'])
```



From here we conclude that IPS panel laptops are more expensive than laptops that do not have IPS panel.

```
# Creating 2 new columns X_rees and Y_res for storing the resolutions separately
new=df['ScreenResolution'].str.split('x',n=1,expand=True)
df['X_res']=new[0]
df['Y_res']=new[1]
df['X_res']=df['X_res'].str.replace(',',").str.findall(r'(\d+\.?\d+)').apply(lambda x:x[0])
df.head()
```

	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory	Gpu	OpSys	Weight	Price	Touchscreen	IPS	X_res	Y_res
0	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 2.3GHz	8	128GB SSD	Intel Iris Plus Graphics 640	macOS	1.37	71378.6832	0	1	2560	1600
1	Apple	Ultrabook	13.3	1440x900	Intel Core i5 1.8GHz	8	128GB Flash Storage	Intel HD Graphics 6000	macOS	1.34	47895.5232	0	0	1440	900
2	HP	Notebook	15.6	Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8	256GB SSD	Intel HD Graphics 620	No OS	1.86	30636.0000	0	0	1920	1080
3	Apple	Ultrabook	15.4	IPS Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16	512GB SSD	AMD Radeon Pro 455	macOS	1.83	135195.3360	0	1	2880	1800
4	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 3.1GHz	8	256GB SSD	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080	0	1	2560	1600

```
df['X_res']=df['X_res'].astype('int')
df['Y_res']=df['Y_res'].astype('int')
df.corr()['Price']
```

```
Inches          0.068197
Ram             0.743007
Weight          0.210370
Price           1.000000
Touchscreen     0.191226
IPS             0.252208
X_res           0.556529
Y_res           0.552809
Name: Price, dtype: float64
```

 # Making a new column PPI(Pixels Per Inch)

df['PPI']=(((df['X_res']**2)+(df['Y_res']**2))**0.5/df['Inches']).astype(float)

df.head()

	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory	Gpu	OpSys	Weight	Price	Touchscreen	IPS	X_res	Y_res	PPI
0	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 2.3GHz	8	128GB SSD	Intel Iris Plus Graphics 640	macOS	1.37	71378.6832	0	1	2560	1600	226.983005
1	Apple	Ultrabook	13.3	1440x900	Intel Core i5 1.8GHz	8	128GB Flash Storage	Intel HD Graphics 6000	macOS	1.34	47895.5232	0	0	1440	900	127.677940
2	HP	Notebook	15.6	Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8	256GB SSD	Intel HD Graphics 620	No OS	1.86	30636.0000	0	0	1920	1080	141.211998
3	Apple	Ultrabook	15.4	IPS Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16	512GB SSD	AMD Radeon Pro 455	macOS	1.83	135195.3360	0	1	2880	1800	220.534624
4	Apple	Ultrabook	13.3	IPS Panel Retina Display 2560x1600	Intel Core i5 3.1GHz	8	256GB SSD	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080	0	1	2560	1600	226.983005

df.corr()['Price']

```
Inches          0.068197
Ram             0.743007
Weight          0.210370
Price           1.000000
Touchscreen     0.191226
IPS             0.252208
X_res           0.556529
Y_res           0.552809
PPI             0.473487
Name: Price, dtype: float64
```

Rather than using X_res,Y_res and Inches column, we will simply use PPI column as it shows high correlation with respect to the price.

 # Dropping the columns that are not required

```
df.drop(columns=['X_res','Y_res','Inches'],inplace=True)
df.head()
```

	Company	TypeName	ScreenResolution	Cpu	Ram	Memory	Gpu	OpSys	Weight	Price	Touchscreen	IPS	PPI
0	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	Intel Core i5 2.3GHz	8	128GB SSD	Intel Iris Plus Graphics 640	macOS	1.37	71378.6832	0	1	226.983005
1	Apple	Ultrabook	1440x900	Intel Core i5 1.8GHz	8	128GB Flash Storage	Intel HD Graphics 6000	macOS	1.34	47895.5232	0	0	127.677940
2	HP	Notebook	Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8	256GB SSD	Intel HD Graphics 620	No OS	1.86	30636.0000	0	0	141.211998
3	Apple	Ultrabook	IPS Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16	512GB SSD	AMD Radeon Pro 455	macOS	1.83	135195.3360	0	1	220.534624
4	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	Intel Core i5 3.1GHz	8	256GB SSD	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080	0	1	226.983005

Feature Engineering on CPU column…

```
df['Cpu'].value_counts()
```

```
Intel Core i5 7200U 2.5GHz       190
Intel Core i7 7700HQ 2.8GHz      146
Intel Core i7 7500U 2.7GHz       134
Intel Core i7 8550U 1.8GHz        73
Intel Core i5 8250U 1.6GHz        72
                                 ...
Intel Core M M3-6Y30 0.9GHz        1
AMD A9-Series 9420 2.9GHz          1
Intel Core i3 6006U 2.2GHz         1
AMD A6-Series 7310 2GHz            1
Intel Xeon E3-1535M v6 3.1GHz      1
Name: Cpu, Length: 118, dtype: int64
```

 # Extracting first 3 words from every row of Cpu column

```
df['CPU']=df['Cpu'].apply(lambda x:" ".join(x.split()[0:3]))
df.head()
```

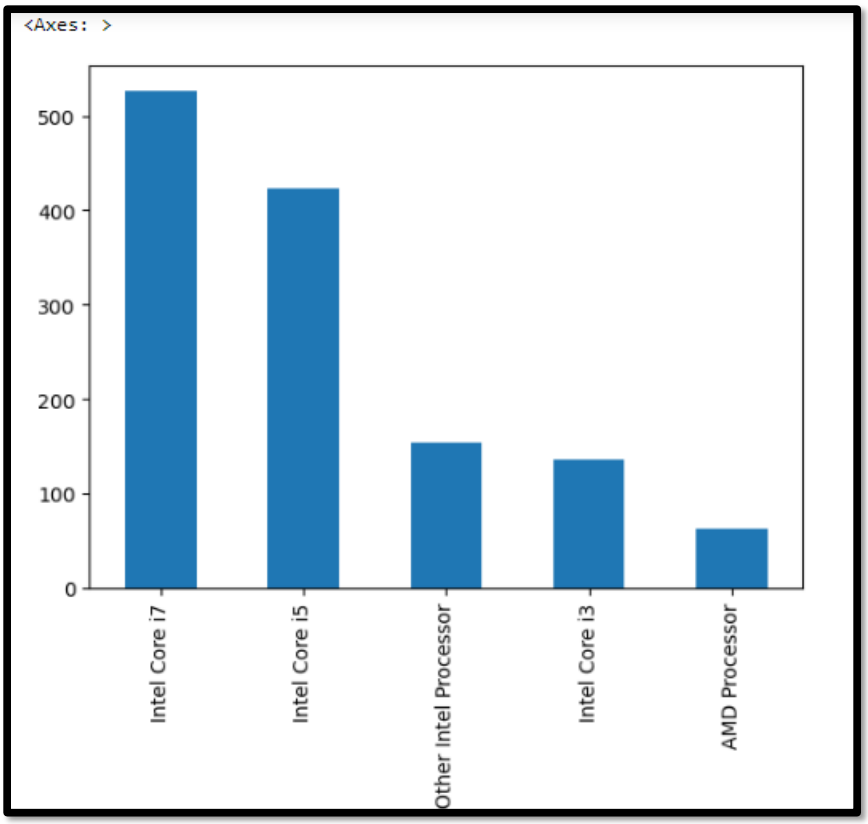
	Company	TypeName	ScreenResolution	Cpu	Ram	Memory	Gpu	OpSys	Weight	Price	Touchscreen	IPS	PPI	CPU
0	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	Intel Core i5 2.3GHz	8	128GB SSD	Intel Iris Plus Graphics 640	macOS	1.37	71378.6832	0	1	226.983005	Intel Core i5
1	Apple	Ultrabook	1440x900	Intel Core i5 1.8GHz	8	128GB Flash Storage	Intel HD Graphics 6000	macOS	1.34	47895.5232	0	0	127.677940	Intel Core i5
2	HP	Notebook	Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8	256GB SSD	Intel HD Graphics 620	No OS	1.86	30636.0000	0	0	141.211998	Intel Core i5
3	Apple	Ultrabook	IPS Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16	512GB SSD	AMD Radeon Pro 455	macOS	1.83	135195.3360	0	1	220.534624	Intel Core i7
4	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	Intel Core i5 3.1GHz	8	256GB SSD	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080	0	1	226.983005	Intel Core i5

 # Categorizing the CPU column on the basis of Brand

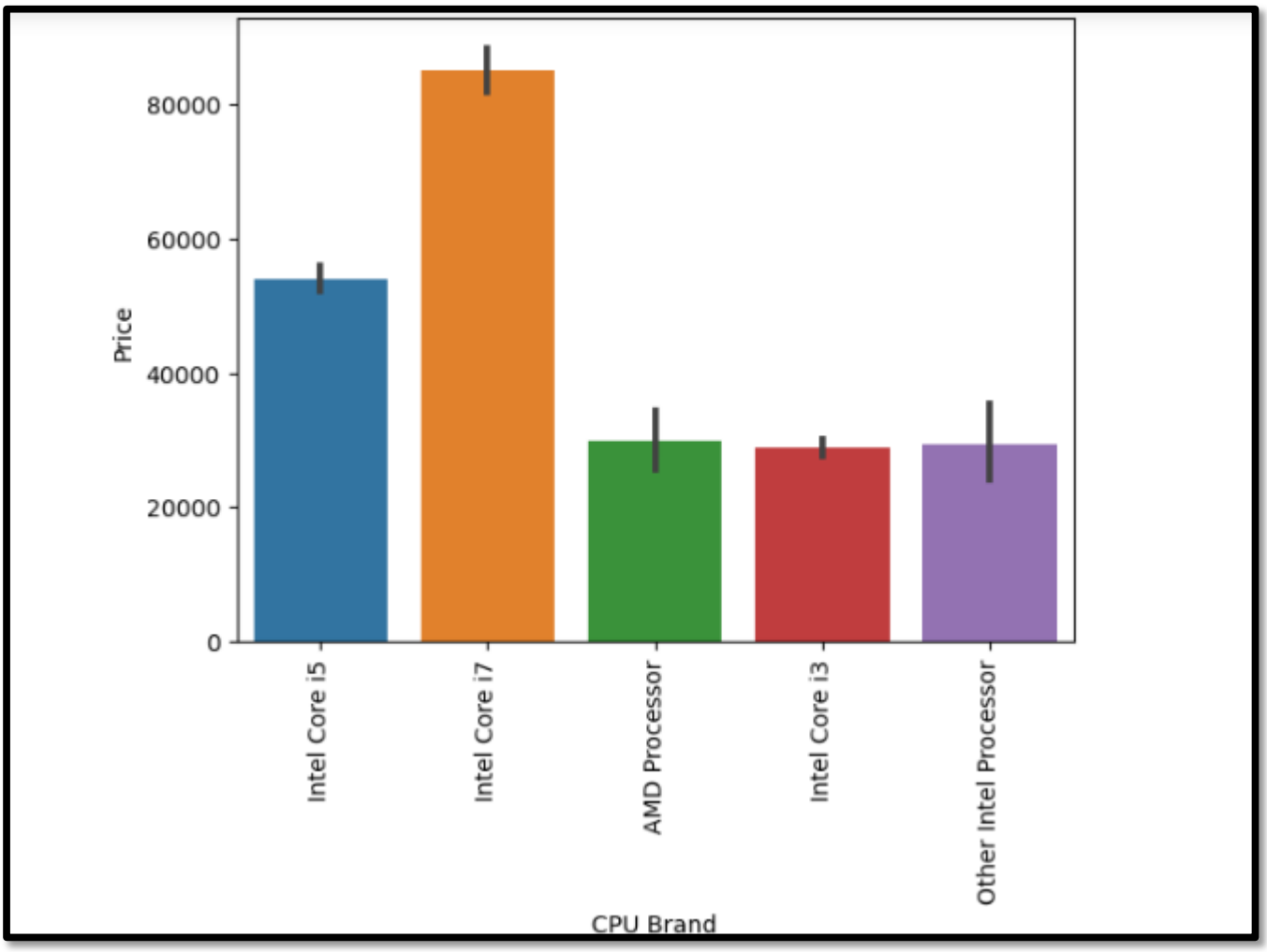
```
def fetch_processor(text):
    if text=='Intel Core i5' or text=='Intel Core i7' or text=='Intel Core i3':
        return text
    else:
        if text.split()[0]=='Intel':
            return 'Other Intel
            Processor'
        else:
            return 'AMD Processor'
df['CPU Brand']=df['CPU'].apply(fetch_processor)
df.head()
```

	Company	TypeName	ScreenResolution	Cpu	Ram	Memory	Gpu	OpSys	Weight	Price	Touchscreen	IPS	PPI	CPU	CPU Brand
0	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	Intel Core i5 2.3GHz	8	128GB SSD	Intel Iris Plus Graphics 640	macOS	1.37	71378.6832	0	1	226.983005	Intel Core i5	Intel Core i5
1	Apple	Ultrabook	1440x900	Intel Core i5 1.8GHz	8	128GB Flash Storage	Intel HD Graphics 6000	macOS	1.34	47895.5232	0	0	127.677940	Intel Core i5	Intel Core i5
2	HP	Notebook	Full HD 1920x1080	Intel Core i5 7200U 2.5GHz	8	256GB SSD	Intel HD Graphics 620	No OS	1.86	30636.0000	0	0	141.211998	Intel Core i5	Intel Core i5
3	Apple	Ultrabook	IPS Panel Retina Display 2880x1800	Intel Core i7 2.7GHz	16	512GB SSD	AMD Radeon Pro 455	macOS	1.83	135195.3360	0	1	220.534624	Intel Core i7	Intel Core i7
4	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	Intel Core i5 3.1GHz	8	256GB SSD	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080	0	1	226.983005	Intel Core i5	Intel Core i5

df['CPU Brand'].value_counts().plot(kind='bar')



```
sns.barplot(x=df['CPU Brand'],y=df['Price'])
plt.xticks(rotation='vertical')
plt.show()
```



```
# dropping unnecessary columns
df.drop(columns=['Cpu','CPU'],inplace=True)
df.head()
```

	Company	TypeName	ScreenResolution	Ram	Memory	Gpu	OpSys	Weight	Price	Touchscreen	IPS	PPI	CPU Brand
0	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	8	128GB SSD	Intel Iris Plus Graphics 640	macOS	1.37	71378.6832	0	1	226.983005	Intel Core i5
1	Apple	Ultrabook	1440x900	8	128GB Flash Storage	Intel HD Graphics 6000	macOS	1.34	47895.5232	0	0	127.677940	Intel Core i5
2	HP	Notebook	Full HD 1920x1080	8	256GB SSD	Intel HD Graphics 620	No OS	1.86	30636.0000	0	0	141.211998	Intel Core i5
3	Apple	Ultrabook	IPS Panel Retina Display 2880x1800	16	512GB SSD	AMD Radeon Pro 455	macOS	1.83	135195.3360	0	1	220.534624	Intel Core i7
4	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	8	256GB SSD	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080	0	1	226.983005	Intel Core i5

```
df['Memory'].value_counts()
df['Memory']=df['Memory'].astype(str).replace('\.0','',regex=True)
df['Memory']=df['Memory'].str.replace('GB','')
df['Memory']=df['Memory'].str.replace('TB','000')
new=df['Memory'].str.split('+',n=1,expand=True)

df['first']=new[0]
df['first']=df['first'].str.strip()

df['second']=new[1]

df['Layer1HDD']=df['first'].apply(lambda x:1 if 'HDD' in x else 0)
df['Layer1SSD']=df['first'].apply(lambda x:1 if 'SSD' in x else 0)
df['Layer1Hybrid']=df['first'].apply(lambda x:1 if 'Hybrid' in x else 0)
df['Layer1Flash_storage']=df['first'].apply(lambda x:1 if 'Flash Storage' in x else 0)
df['first']=df['first'].str.replace(r'\D','')
df['second'].fillna('0',inplace=True)
```

```
df['Layer2HDD']=df['second'].apply(lambda x:1 if 'HDD' in x else 0)
df['Layer2SSD']=df['second'].apply(lambda x:1 if 'SSD' in x else 0)
df['Layer2Hybrid']=df['second'].apply(lambda x:1 if 'Hybrid' in x else 0)
df['Layer2Flash_storage']=df['second'].apply(lambda x:1 if 'Flash Storage' in x else 0)
df['second']=df['first'].str.replace(r'\D','')
df['first']=df['first'].astype(int)
df['second']=df['second'].astype(int)

df['HDD']=(df['first']*df['Layer1HDD']+df['second']*df['Layer2HDD'])
df['SSD']=(df['first']*df['Layer1SSD']+df['second']*df['Layer2SSD'])
df['Hybrid']=(df['first']*df['Layer1Hybrid']+df['second']*df['Layer2Hybrid'])
df['Flash_storage']=(df['first']*df['Layer1Flash_storage']+df['second']*df['Layer2Flash_storage'])

df.drop(columns=['first','second','Layer1HDD','Layer1SSD','Layer1Hybrid','Layer1Flash_storage',
                 'Layer2HDD','Layer2SSD','Layer2Hybrid','Layer2Flash_storage'],inplace=True)
df.head()
```

```
256GB SSD                          412
1TB HDD                            223
500GB HDD                          132
512GB SSD                          118
128GB SSD +  1TB HDD                94
128GB SSD                           76
256GB SSD +  1TB HDD                73
32GB Flash Storage                  38
2TB HDD                             16
64GB Flash Storage                  15
512GB SSD +  1TB HDD                14
1TB SSD                             14
256GB SSD +  2TB HDD                10
1.0TB Hybrid                         9
256GB Flash Storage                  8
16GB Flash Storage                   7
32GB SSD                             6
180GB SSD                            5
128GB Flash Storage                  4
512GB SSD +  2TB HDD                 3
16GB SSD                             3
512GB Flash Storage                  2
1TB SSD +  1TB HDD                   2
256GB SSD +  500GB HDD               2
128GB SSD +  2TB HDD                 2
256GB SSD +  256GB SSD               2
512GB SSD +  256GB SSD               1
512GB SSD +  512GB SSD               1
64GB Flash Storage +  1TB HDD        1
1TB HDD +  1TB HDD                   1
32GB HDD                             1
64GB SSD                             1
128GB HDD                            1
```

Company	TypeName	ScreenResolution	Ram	Memory	Gpu	OpSys	Weight	Price	Touchscreen	IPS	PPI	CPU Brand	HDD	SSD	Hybrid	Flash
Apple	Ultrabook	IPS Panel Retina Display 2560x1600	8	128 SSD	Intel Iris Plus Graphics 640	macOS	1.37	71378.6832	0	1	226.983005	Intel Core i5	0	128	0	
Apple	Ultrabook	1440x900	8	128 Flash Storage	Intel HD Graphics 6000	macOS	1.34	47895.5232	0	0	127.677940	Intel Core i5	0	0	0	
HP	Notebook	Full HD 1920x1080	8	256 SSD	Intel HD Graphics 620	No OS	1.86	30636.0000	0	0	141.211998	Intel Core i5	0	256	0	
Apple	Ultrabook	IPS Panel Retina Display 2880x1800	16	512 SSD	AMD Radeon Pro 455	macOS	1.83	135195.3360	0	1	220.534624	Intel Core i7	0	512	0	
Apple	Ultrabook	IPS Panel Retina Display 2560x1600	8	256 SSD	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080	0	1	226.983005	Intel Core i5	0	256	0	

df.drop(columns=['Memory'],inplace=True)

 # checking the correlation to find the columns that are strongly and weakly correlated to the price

df.corr()['Price']

```
Ram               0.743007
Weight            0.210370
Price             1.000000
Touchscreen       0.191226
IPS               0.252208
PPI               0.473487
HDD              -0.303340
SSD               0.668854
Hybrid           -0.020186
Flash_storage    -0.040511
Name: Price, dtype: float64
```

So, from here we conclude that HDD and SSD columns are strongly correlated and Hybrid and Flash_storage are weakly correlated with the price.

 # dropping Hybrid and Flash_storage columns as they are not so necessary

df.drop(columns=['Hybrid','Flash_storage'],inplace=True)

df.head()

	Company	TypeName	ScreenResolution	Ram	Gpu	OpSys	Weight	Price	Touchscreen	IPS	PPI	CPU Brand	HDD	SSD
0	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	8	Intel Iris Plus Graphics 640	macOS	1.37	71378.6832	0	1	226.983005	Intel Core i5	0	128
1	Apple	Ultrabook	1440x900	8	Intel HD Graphics 6000	macOS	1.34	47895.5232	0	0	127.677940	Intel Core i5	0	0
2	HP	Notebook	Full HD 1920x1080	8	Intel HD Graphics 620	No OS	1.86	30636.0000	0	0	141.211998	Intel Core i5	0	256
3	Apple	Ultrabook	IPS Panel Retina Display 2880x1800	16	AMD Radeon Pro 455	macOS	1.83	135195.3360	0	1	220.534624	Intel Core i7	0	512
4	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	8	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080	0	1	226.983005	Intel Core i5	0	256

Transforming the Gpu column

df['Gpu'].value_counts()

```
Intel HD Graphics 620      281
Intel HD Graphics 520      185
Intel UHD Graphics 620      68
Nvidia GeForce GTX 1050     66
Nvidia GeForce GTX 1060     48
                          ...
AMD Radeon R5 520            1
AMD Radeon R7                1
Intel HD Graphics 540        1
AMD Radeon 540               1
ARM Mali T860 MP4            1
Name: Gpu, Length: 110, dtype: int64
```

 # Extracting the brand name

```
df['Gpu_brand']=df['Gpu'].apply(lambda
x:x.split()[0])
```

```
df.head()
```

	Company	TypeName	ScreenResolution	Ram	Gpu	OpSys	Weight	Price	Touchscreen	IPS	PPI	CPU Brand	HDD	SSD	Gpu_brand
0	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	8	Intel Iris Plus Graphics 640	macOS	1.37	71378.6832	0	1	226.983005	Intel Core i5	0	128	Intel
1	Apple	Ultrabook	1440x900	8	Intel HD Graphics 6000	macOS	1.34	47895.5232	0	0	127.677940	Intel Core i5	0	0	Intel
2	HP	Notebook	Full HD 1920x1080	8	Intel HD Graphics 620	No OS	1.86	30636.0000	0	0	141.211998	Intel Core i5	0	256	Intel
3	Apple	Ultrabook	IPS Panel Retina Display 2880x1800	16	AMD Radeon Pro 455	macOS	1.83	135195.3360	0	1	220.534624	Intel Core i7	0	512	AMD
4	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	8	Intel Iris Plus Graphics 650	macOS	1.37	96095.8080	0	1	226.983005	Intel Core i5	0	256	Intel

```
df['Gpu_brand'].value_counts()
```

```
Intel     722
Nvidia    400
AMD       180
ARM         1
Name: Gpu_brand, dtype: int64
```

 # dropping the row with ARM Gpu

```
df=df[df['Gpu_brand']!='ARM']
df['Gpu_brand'].value_counts()
```

	Company	TypeName	ScreenResolution	Ram	OpSys	Weight	Price	Touchscreen	IPS	PPI	CPU Brand	HDD	SSD	Gpu_brand
0	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	8	macOS	1.37	71378.6832	0	1	226.983005	Intel Core i5	0	128	Intel
1	Apple	Ultrabook	1440x900	8	macOS	1.34	47895.5232	0	0	127.677940	Intel Core i5	0	0	Intel
2	HP	Notebook	Full HD 1920x1080	8	No OS	1.86	30636.0000	0	0	141.211998	Intel Core i5	0	256	Intel
3	Apple	Ultrabook	IPS Panel Retina Display 2880x1800	16	macOS	1.83	135195.3360	0	1	220.534624	Intel Core i7	0	512	AMD
4	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	8	macOS	1.37	96095.8080	0	1	226.983005	Intel Core i5	0	256	Intel

Transforming the OpSys column

 # categorising OpSys column into 3 different categories

```
def cat_os(inp):
    if inp=='Windows 10' or inp=='Windows 7' or inp=='Windows 10 S':
        return 'Windows'
    elif inp=='macOS' or inp=='Mac OS X':
        return 'Mac'
    else:
         return
         'NoOS/Linux/Others'
         df['os']=df['OpSys'].apply(cat_o)
 df.head()
```

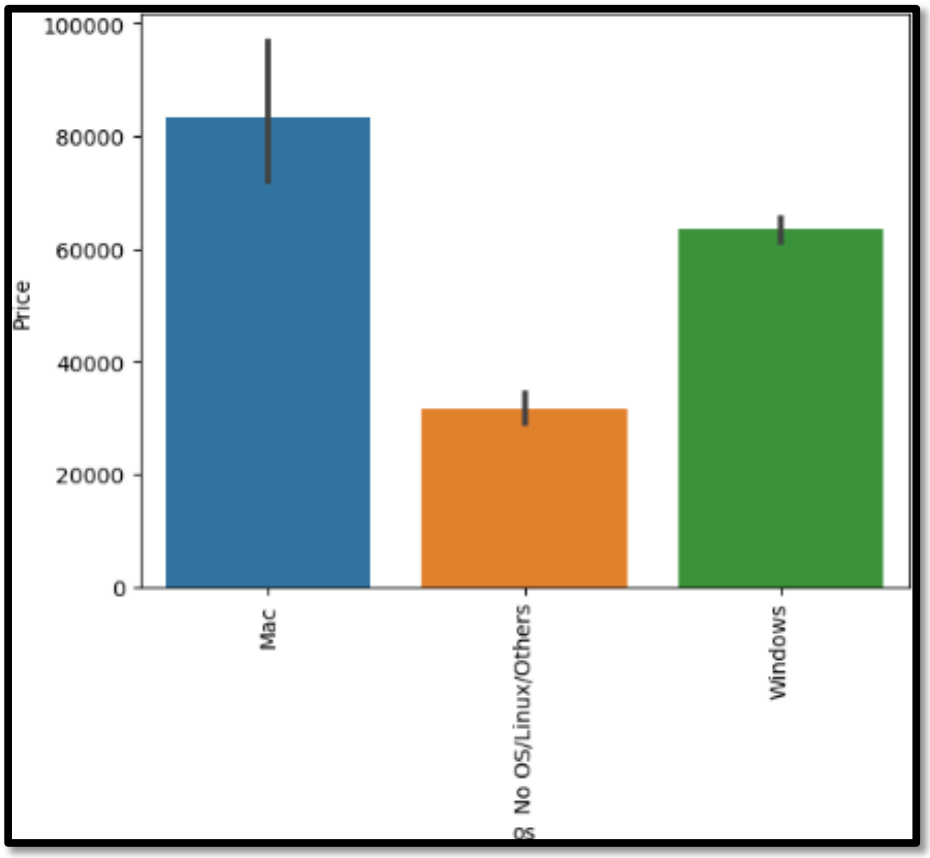
	Company	TypeName	ScreenResolution	Ram	OpSys	Weight	Price	Touchscreen	IPS	PPI	CPU Brand	HDD	SSD	Gpu_brand	os
0	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	8	macOS	1.37	71378.6832	0	1	226.983005	Intel Core i5	0	128	Intel	Mac
1	Apple	Ultrabook	1440x900	8	macOS	1.34	47895.5232	0	0	127.677940	Intel Core i5	0	0	Intel	Mac
2	HP	Notebook	Full HD 1920x1080	8	No OS	1.86	30636.0000	0	0	141.211998	Intel Core i5	0	256	Intel	No OS/Linux/Others
3	Apple	Ultrabook	IPS Panel Retina Display 2880x1800	16	macOS	1.83	135195.3360	0	1	220.534624	Intel Core i7	0	512	AMD	Mac
4	Apple	Ultrabook	IPS Panel Retina Display 2560x1600	8	macOS	1.37	96095.8080	0	1	226.983005	Intel Core i5	0	256	Intel	Mac

```
df['OpSys'].value_counts()
```

```
Windows 10      1072
No OS             66
Linux             62
Windows 7         45
Chrome OS         26
macOS             13
Mac OS X           8
Windows 10 S       8
Android            2
Name: OpSys, dtype: int64
```

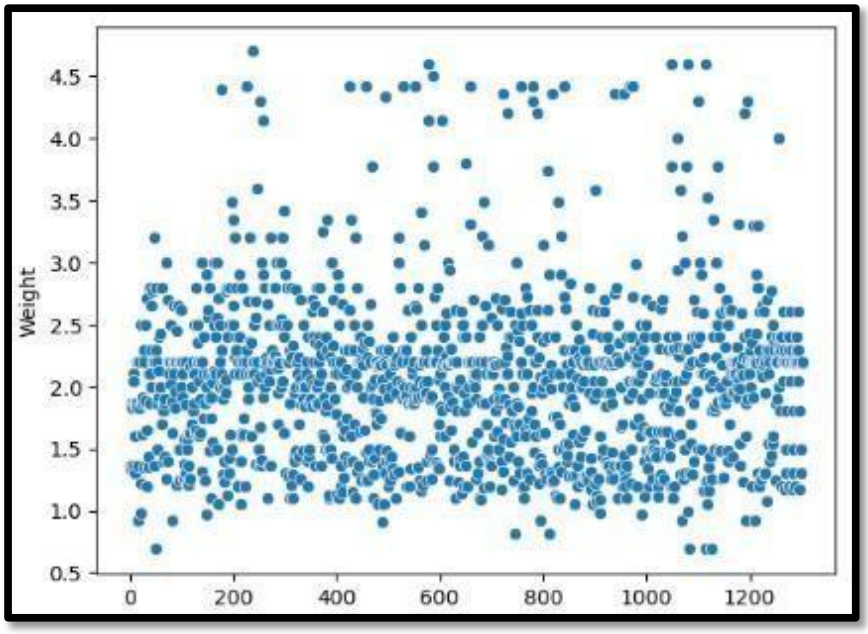
 # dropping the OpSys column

```
df.drop(columns=['OpSys'],inplace=True)
sns.barplot(x=df['os'],y=df['Price'])
plt.xticks(rotation='vertical')

plt.show()
```



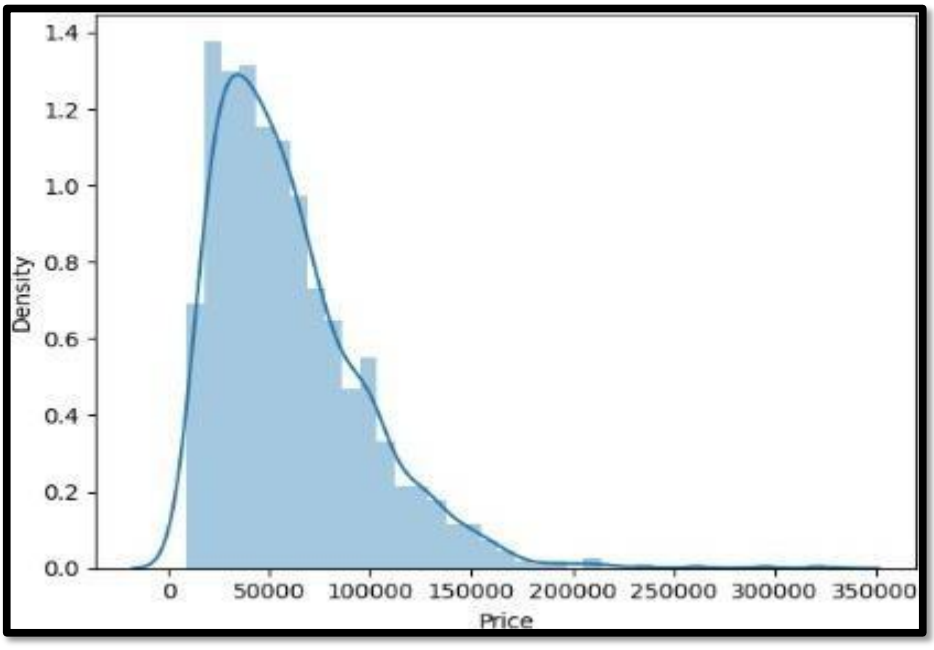
Analyzing the Weight column

sns.scatterplot(df['Weight'])



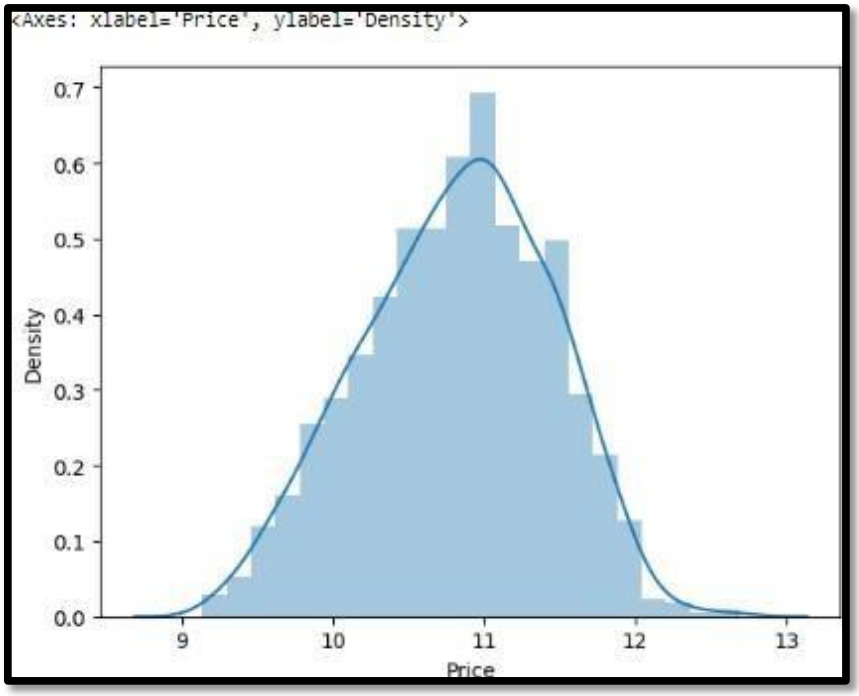
Removing the skewness of the price column

sns.distplot(df['Price'])



 # applying the log function

```
sns.distplot(np.log(df['Price']))
```




```
df.drop(columns=['ScreenResolution'],inplace=True)
```

Model Building

splitting the data in x and y

x=df.drop(columns=['Price'])

y=np.log(df['Price'])

X

	Company	TypeName	Ram	Weight	Touchscreen	IPS	PPI	CPU Brand	HDD	SSD	Gpu_brand	os
0	Apple	Ultrabook	8	1.37	0	1	226.983005	Intel Core i5	0	128	Intel	Mac
1	Apple	Ultrabook	8	1.34	0	0	127.677940	Intel Core i5	0	0	Intel	Mac
2	HP	Notebook	8	1.86	0	0	141.211998	Intel Core i5	0	256	Intel	No OS/Linux/Others
3	Apple	Ultrabook	16	1.83	0	1	220.534624	Intel Core i7	0	512	AMD	Mac
4	Apple	Ultrabook	8	1.37	0	1	226.983005	Intel Core i5	0	256	Intel	Mac
...	...	...	...	...	...	...	...	...	...	...	...	...
1298	Lenovo	2 in 1 Convertible	4	1.80	1	1	157.350512	Intel Core i7	0	128	Intel	Windows
1299	Lenovo	2 in 1 Convertible	16	1.30	1	1	276.053530	Intel Core i7	0	512	Intel	Windows
1300	Lenovo	Notebook	2	1.50	0	0	111.935204	Other Intel Processor	0	0	Intel	Windows
1301	HP	Notebook	6	2.19	0	0	100.454670	Intel Core i7	1000	0	AMD	Windows
1302	Asus	Notebook	4	2.20	0	0	100.454670	Other Intel Processor	500	0	Intel	Windows

1302 rows × 12 columns

Y

```
0       11.175755
1       10.776777
2       10.329931
3       11.814476
4       11.473101
          ...
1298    10.433899
1299    11.288115
1300     9.409283
1301    10.614129
1302     9.886358
Name: Price, Length: 1302, dtype: float64
```

 # splitting traiined and tested

```
from  sklearn.model_selection  import  train_test_split
x_train, x_test, y_train, y_test= train_test_split(x,y,test_size=0.15,random_state=2)
x_train
```

	Company	TypeName	Ram	Weight	Touchscreen	IPS	PPI	CPU Brand	HDD	SSD	Gpu_brand	os
183	Toshiba	Notebook	8	2.00	0	0	100.454670	Intel Core i5	0	128	Intel	Windows
1141	MSI	Gaming	8	2.40	0	0	141.211998	Intel Core i7	128	128	Nvidia	Windows
1049	Asus	Netbook	4	1.20	0	0	135.094211	Other Intel Processor	0	0	Intel	No OS/Linux/Others
1020	Dell	2 in 1 Convertible	4	2.08	1	1	141.211998	Intel Core i3	1000	0	Intel	Windows
878	Dell	Notebook	4	2.18	0	0	141.211998	Intel Core i5	128	128	Nvidia	Windows
...	...	...	...	...	...	...	...	...	...	...	...	...
466	Acer	Notebook	4	2.20	0	0	100.454670	Intel Core i3	500	0	Nvidia	Windows
299	Asus	Ultrabook	16	1.63	0	0	141.211998	Intel Core i7	0	512	Nvidia	Windows
493	Acer	Notebook	8	2.20	0	0	100.454670	AMD Processor	1000	0	AMD	Windows
527	Lenovo	Notebook	8	2.20	0	0	100.454670	Intel Core i3	2000	0	Nvidia	No OS/Linux/Others
1193	Apple	Ultrabook	8	0.92	0	1	226.415547	Other Intel Processor	0	0	Intel	Mac

 # handling the columns with string values

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import
KNeighborsRegressorfrom sklearn.tree import
DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score,mean_absolute_error
```

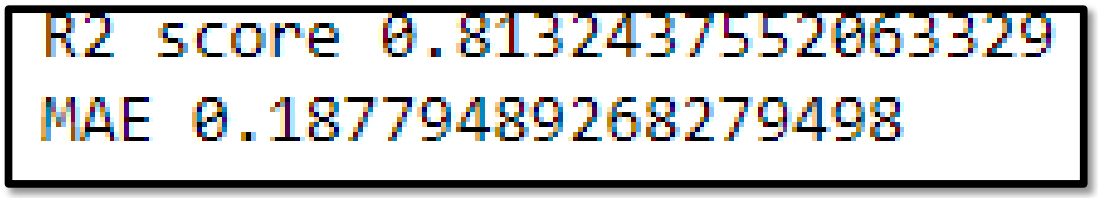
Linear Regression

```
step1=ColumnTransformer(transformers=[
    ('col_tnf',OneHotEncoder(sparse=False,drop='first'),[0,1,7,10,11])
],remainder='passthrough')
step2=LinearRegression()
pipe=Pipeline([
    ('step1',step1),
    ('step2',step2)
])

pipe.fit(x_train,y_train)
y_pred=pipe.predict(x_test)

print('R2 Score:',r2_score(y_test,y_pred))
print('MAE:',mean_absolute_error(y_test,y_pred))
```

```
R2 Score: 0.8081840565215 39
MAE: 0.20987730547745137
```

KNN

```
step1 = ColumnTransformer(transformers=[
   ('col_tnf',OneHotEncoder(sparse=False,drop='first'),[0,1,7,10,11])
],remainder='passthrough')
step2 =
KNeighborsRegressor(n_neighbors=3)pipe =


Pipeline([
   ('step1',step1),
   ('step2',step2)
])

pipe.fit(x_train,y_train)
y_pred = pipe.predict(x_test)
print('R2 score',r2_score(y_test,y_pred))
print('MAE',mean_absolute_error(y_test,y_pred))
```



Decision Tree

```
step1 = ColumnTransformer(transformers=[
   ('col_tnf',OneHotEncoder(sparse=False,drop='first'),[0,1,7,10,11])
],remainder='passthrough')
step2 =
DecisionTreeRegressor(max_depth=8)pipe =

Pipeline([
   ('step1',step1),
   ('step2',step2)
])

pipe.fit(x_train,y_train)
y_pred = pipe.predict(x_test)
print('R2 score',r2_score(y_test,y_pred))
print('MAE',mean_absolute_error(y_test,y_pred))
```

```
R2 score 0.8367157654222608
MAE 0.18477848008006454
```

Random Forest

```
step1 = ColumnTransformer(transformers=[
   ('col_tnf',OneHotEncoder(sparse=False,drop='first'),[0,1,7,10,11])
],remainder='passthrough')

step2   =   RandomForestRegressor(n_estimators=100,
                        random_state=3,
                        max_samples=0.5,
                        max_features=0.75,
                        max_depth=15)

pipe =
   Pipeline([
   ('step1',step1),
   ('step2',step2)
])

pipe.fit(x_train,y_train)
y_pred = pipe.predict(x_test)

print('R2 score',r2_score(y_test,y_pred))
print('MAE',mean_absolute_error(y_test,y_pred))
```

```
R2 score 0.8868842180893174
MAE 0.1582963776323844 7
```

So, from here we conclude that Random Forest Regressor model is the best model giving highest R2 score.

Exporting the Model

```
import pickle

pickle.dump(df,open('df.pkl','wb'))
pickle.dump(pipe,open('pipe.pkl','wb'))
```

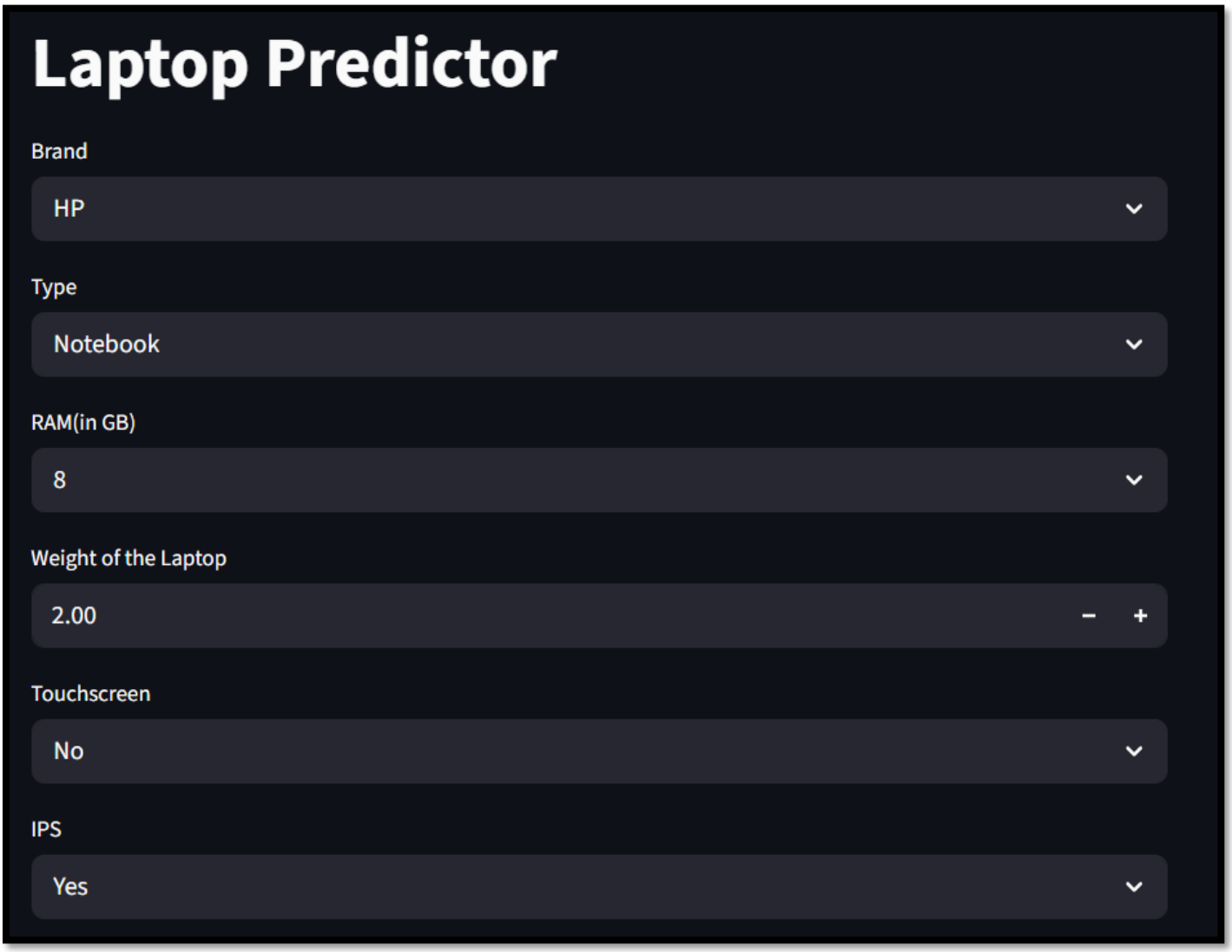
Creating a Web Interface:

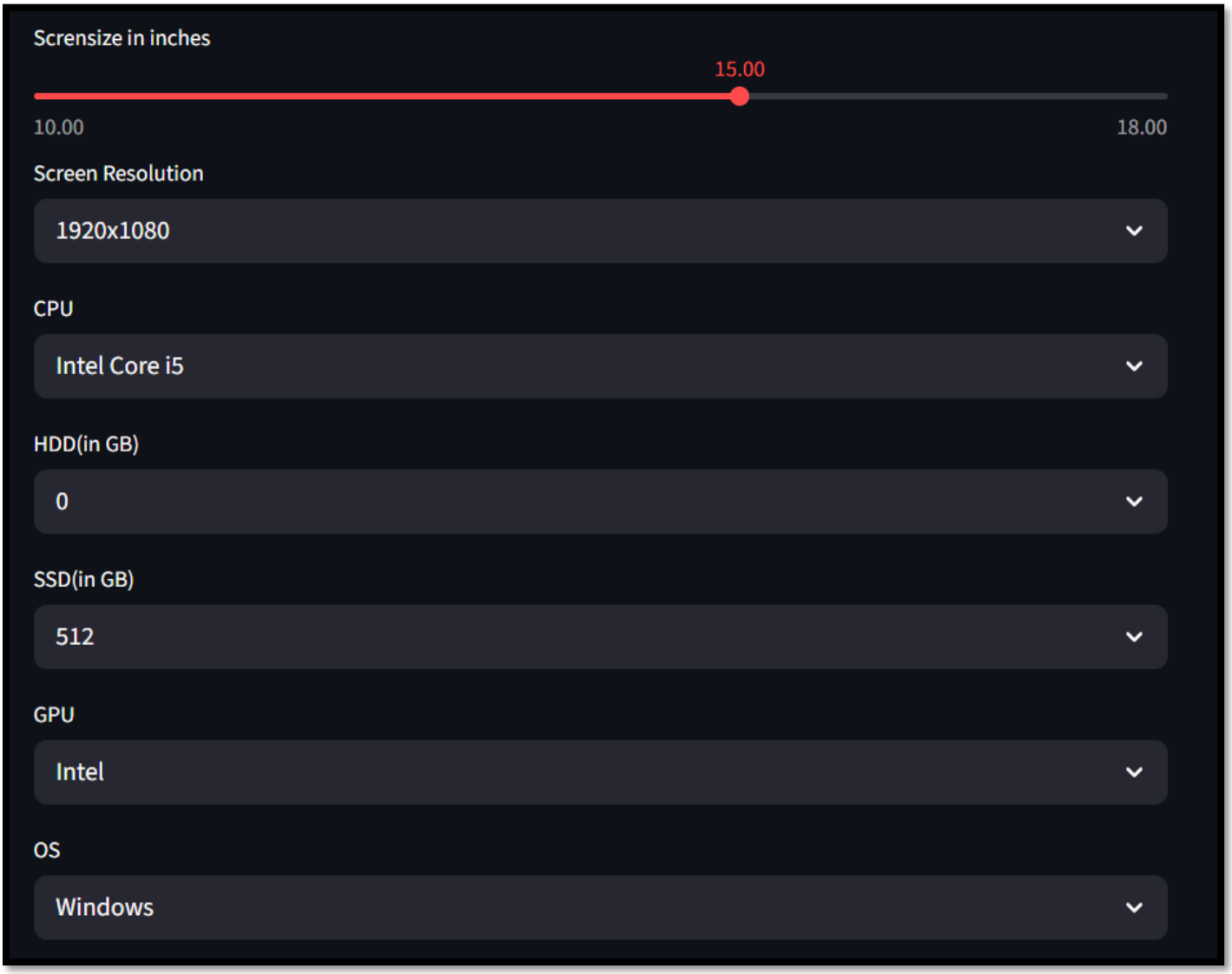
app.py-

```
import streamlit as st
import pickle
import numpy as np

# import the model
pipe = pickle.load(open('pipe.pkl','rb'))
df = pickle.load(open('df.pkl','rb'))

st.title("Laptop Predictor")

# brand
company = st.selectbox('Brand',df['Company'].unique())

# type of laptop
type = st.selectbox('Type',df['TypeName'].unique())

# Ram
ram = st.selectbox('RAM(in GB)',[2,4,6,8,12,16,24,32,64])

# weight
weight = st.number_input('Weight of the Laptop')

# Touchscreen
touchscreen = st.selectbox('Touchscreen',['No','Yes'])

# IPS
ips = st.selectbox('IPS',['No','Yes'])

# screen size
screen_size = st.slider('Scrensize in inches', 10.0, 18.0, 13.0)

# resolution
resolution = st.selectbox('Screen
```

```
Resolution',['1920x1080','1366x768','1600x900','3840x2160','3200x1800','2880x1800','2560x1600','2560x1440','2304
x1440'])

#cpu
cpu = st.selectbox('CPU',df['Cpu brand'].unique())

hdd = st.selectbox('HDD(in GB)',[0,128,256,512,1024,2048])

ssd = st.selectbox('SSD(in GB)',[0,8,128,256,512,1024])

gpu = st.selectbox('GPU',df['Gpu brand'].unique())

os = st.selectbox('OS',df['os'].unique())

if st.button('Predict Price'):
    # query
    ppi = None
    if touchscreen == 'Yes':
        touchscreen = 1
    else:
        touchscreen = 0

    if ips == 'Yes':
        ips = 1
    else:
        ips = 0

    X_res = int(resolution.split('x')[0])
    Y_res = int(resolution.split('x')[1])
    ppi = ((X_res**2) + (Y_res**2))**0.5/screen_size
    query = np.array([company,type,ram,weight,touchscreen,ips,ppi,cpu,hdd,ssd,gpu,os])

    query = query.reshape(1,12)
    st.title("The predicted price of this configuration is " + str(int(np.exp(pipe.predict(query)[0]))))
```

Output:
run app.py





App link: https://render-demo-lpp.onrender.com/

6.4 Conclusion & Future scope:

Conclusion:

Our Laptop Price Prediction project has successfully addressed the challenge of accurately estimating Laptop prices to satisfy both buyers and sellers while providing valuable insights for market analysts. This project's journey began with the analysis of a diverse dataset containing information about Laptops from various manufacturers, their prices, and key features like Processor, GPU, and more.

Through meticulous data analysis, pre-processing, and visualization, we gained a comprehensive understanding of the dataset's characteristics. We then leveraged the power of machine learning, specifically the Random Forest model, to build a robust predictive system.

Future Scope for modification:

1. Advanced Feature Engineering: Explore additional features that influence Laptop prices, such as regional factors, economic indicators, and seasonal trends. Refine feature selection to enhance prediction accuracy.
2. Dynamic Pricing Model: Implement a dynamic pricing system that takes real-time market data into account, allowing users to get up-to-the-minute price estimates.
3. User Preferences and History: Develop a feature that considers user preferences and their search history to offer personalized Laptop price predictions. This enhances the user experience and provides more accurate recommendations.
4. Market Analysis Tools: Offer advanced market analysis features to assist users in understanding the dynamics of the Laptop market better. Features could include trend analysis, market forecasts, and data visualization tools.
5. Scalability and Performance: Focus on performance optimization to ensure the system remains responsive as the user base and data volume grow. Consider cloud-based solutions for scalability.

6.5 References

References: https://github.com/Sagar-Gupta008/lpp-sagar